

Operating Systems

No. 5

Sarun Intakosum

Interprocess Communication (IPC)

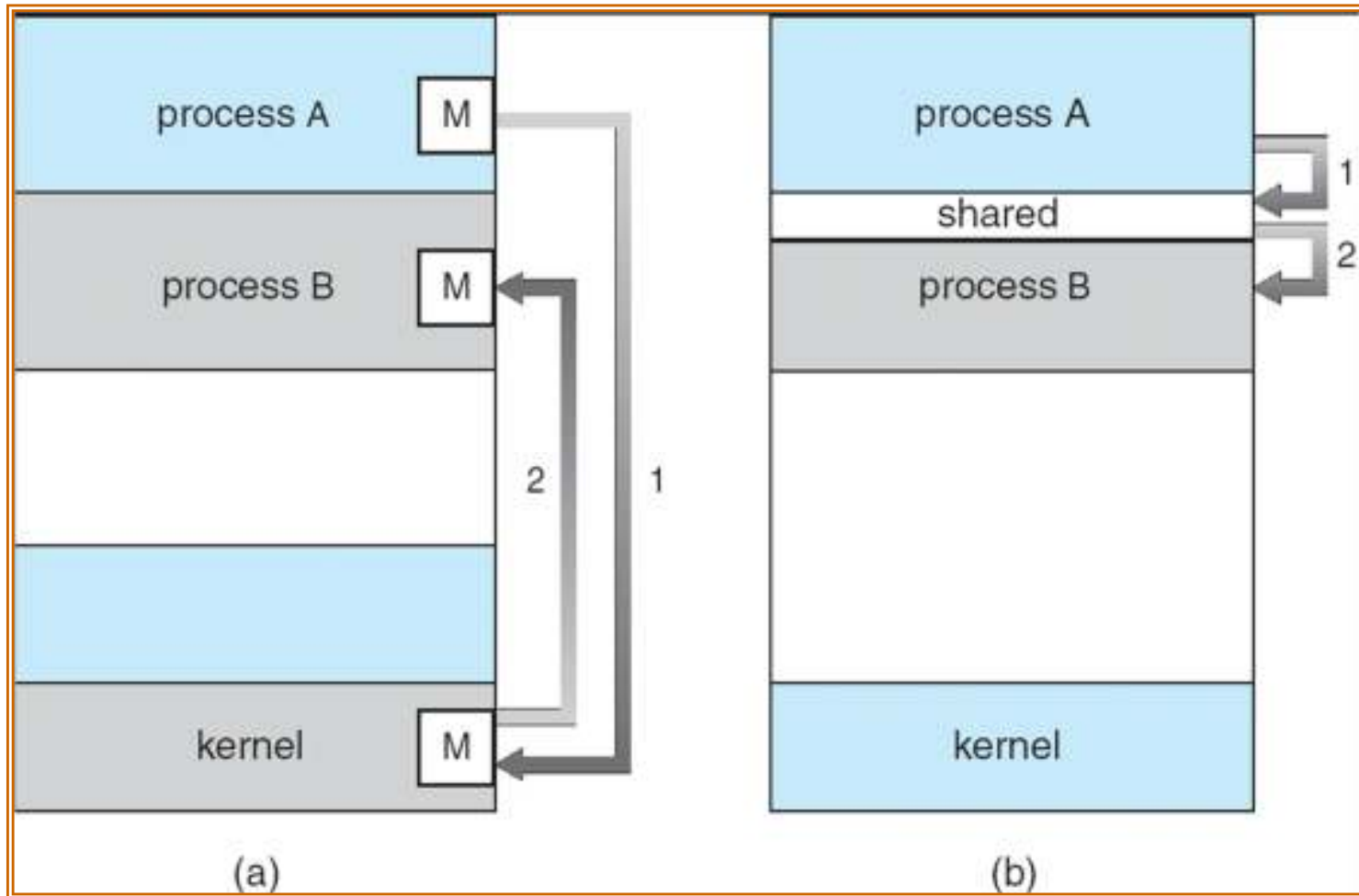
Interprocess Communication

- Processes within a system may be independent or cooperating
- Cooperative processes can influence or be influenced by other processes, including data exchange.
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
- Cooperating processes need **interprocess communication (IPC)**
- Three models of IPC
 - Signal
 - Shared memory
 - Message passing

Signals

- In an operating system, a signal is a way for one program (or even the OS itself) to interrupt another program.
- Examples:
 - Errors or Exceptions: "Hey, something went wrong here!" (e.g., trying to divide by zero)
 - External Events: "The user just pressed Ctrl+C!" (requesting the program to stop)
 - Notifications: "Child process finished!" (a message from a child to its parent)
 - Alarms: "Time's up!" (after a set period)
- So signal is a **software interrupts** that notify a process that an event has occurred
- Signals carry minimal information (typically just the signal number), so they are often used for simple notifications.
- Processes may catch, use OS default, or mask a signal
 - To catch a signal, one must define a specific routine that the operating system invokes when it provides the signal.
 - Use operating system's default action to handle the signal
 - Masking a signal involves directing the operating system to withhold signals of a specific type until the process removes the signal mask.

Communications Models



UNIX Interprocess Communications

- Signal
- Pipe
- Shared memory

Signal

- **signal.h** is a header file in the C Standard Library, specifically designed for handling signals in Unix-like operating systems.
- It provides definitions and declarations necessary for signal handling, allowing programs to interact with and respond to signals.
- All signal names (that represent signal number) start with SIG.

Signal (Cont.)

- SIGFPE An arithmetic exception (e.g., division by zero, overflow and underflow) has occurred.
- SIGINT An interrupt (e.g., Ctrl-C) has occurred.
- SIGTERM Software termination.
- SIGQUIT (non-ANSI C) Quit execution (e.g., Ctrl-\\).
- SIGUSR1 and SIGUSR2 (e.g., non-ANSI C) User-defined signal 1 and 2.
- SIGKILL (non-ANSI C) Kill the process (cannot be caught or ignored)

Handling Signals: Function `signal()`

- The most important function for handling signals is `signal()`.
- `signal()`:
 - Sets a function as the handler for a specific signal.

signal() (Contd.)

- `signal(int signum, void (*handler)(int))`
- `signal()` requires two arguments:
 - The first one is an integer that is a signal number. In practice, we always pass the signal names discussed earlier.
 - The second one is the pointer to a signal handler function. For simplicity, you can think of it as the function's name
 - However, the signal handler function must be a function that:
 - Accepts an integer argument and returns nothing. The integer argument is a signal number.
- In summary, the `signal()` function tells the OS that if it detects the signal specified in the first argument, the signal handler specified in the second argument is called.
- `sig_demo.c`

Handling Multiple Signal Types

- There are two choices:
 - Install a signal handler for each signal.

```
signal(SIGINT, int_handler);  
signal(SIGQUIT, quit_handler);
```

```
...
```

```
void int_handler(int sig)
```

```
{
```

```
.....
```

```
}
```

```
void quit_handler(int sig)
```

```
{
```

```
.....
```

```
}
```

Handling Multiple Signal Types (Contd.)

- Create a single handler whose only input is the signal to be sent:

```
signal(SIGINT, sig_handler);  
signal(SIGALRM, sig_handler);
```

```
void sig_handler(int sig)  
{  
    .....  
    switch (sig) {  
        case SIGINT : .....  
        case SIGALRM : ....  
    }  
}
```

multiple_sig.c

Sending a Signal: raise() function

- raise() function is used to send a signal to the **executing process itself**.
- The following is the function prototype:
 - `int raise(int sig)`
 - Where sig is a signal number to be sent to the current process. This can be any valid signal number defined in `signal.h`, such as `SIGINT`, `SIGTERM`, etc.
 - The `raise()` function returns 0 on success.
 - If an error occurs, it returns a nonzero value.
- When `raise(sig)` is called:
 - The signal specified by sig is sent to the calling process.
 - The process's signal handler for that signal (if one is set) is invoked.
 - If no custom handler is set, the default action associated with the signal is performed.
 - This could be terminating the process, ignoring the signal, or stopping/continuing execution.
- **To send a signal to other processes, the system call `kill()` should be used.**
- `raise.c`

Sending a Signal to Another Process: System Call kill()

- The kill() function in C is used to send a signal to a specified process or a group of processes.
- This function is typically used for inter-process communication, allowing **one process to notify another process of certain events** or to **control its execution (e.g., terminate it)**.
- The following is the prototype of kill():
 - `int kill(pid_t pid, int sig)`
 - `pid =>` The process ID or group of processes to which the signal will be sent.
 - Positive value: Sends the signal to the process with the specified PID.
 - 0: Sends the signal to all processes in the same process group as the calling process.
 - -1: Sends the signal to all processes for which the calling process has permission to send signals, except for process 1 (init).
 - Negative value (other than -1): Sends the signal to all processes whose process group ID is equal to the absolute value of the pid.
 - `sig =>` The signal number to be sent.
 - The kill() function returns 0 on success.
 - If an error occurs, it returns -1
- **kill_demo.c**

kill command

- The kill command can be used to send any signal to a process.
- The kill command has the following form:
 - kill **-signal** pid pid pid ... pid
- The first argument, shown in boldface, is the signal name that represents the signal to be sent.
- This signal name does not have to include SIG. For example, if you want to send signals SIGINT you can use INT.
- You can also use signal number
- One signal number that you should know is 9 which is SIGKILL.
- Use *kill -9 pid* to kill a process that is so stubborn.
- SIGKILL is a signal that the process cannot handle or ignore; therefore, the process that got SIGKILL must be terminated.
- If the signal name is missing, the default SIGTERM is used.

Message Passing

- Message-based interprocess communication
- IPC facility provides two operations:
 - **send**(*message*) – message size fixed or variable
 - **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
 - establish a *communication link* between them
 - exchange messages via send/receive
- Messages can be passed in one direction at a time
 - One process is the sender and the other is the receiver
- Message passing can be bidirectional
 - Each process can act as either a sender or a receiver
- Popular implementation is a pipe
 - A region of memory protected by the OS that serves as a buffer, allowing two or more processes to exchange data

Synchronization

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
 - **Blocking send** has the sender block until the message is received
 - **Blocking receive** has the receiver block until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send** has the sender send the message and continue
 - **Non-blocking receive** has the receiver receive a valid message or null

Pipes

- An anonymous pipe (we will call pipe for short) is used for one-way communication of a stream of bytes.
- The command to create a pipe is `pipe`, which takes an array of two integers. It fills in the array with two file descriptors that can be used for low-level I/O.

Creating a Pipe

- `int pfd[2];`
- `pipe(pfd);`



`pfd[1]`

`pfd[0]`

write to
this end

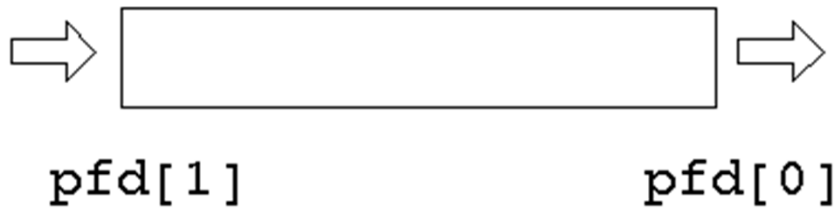
read from
this end

I/O with a pipe

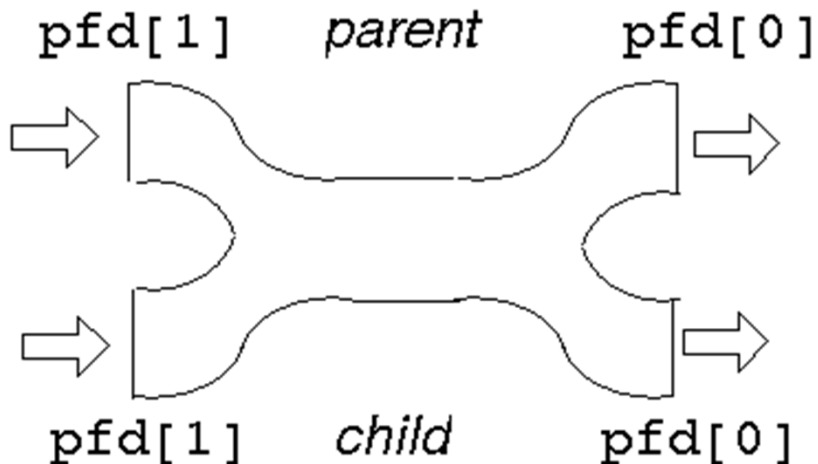
- These two file descriptors can be used for block I/O
 - `write(pfd[1], buf, size);`
 - `read(pfd[0], buf, SIZE);`

Fork and a pipe ()

Before
fork



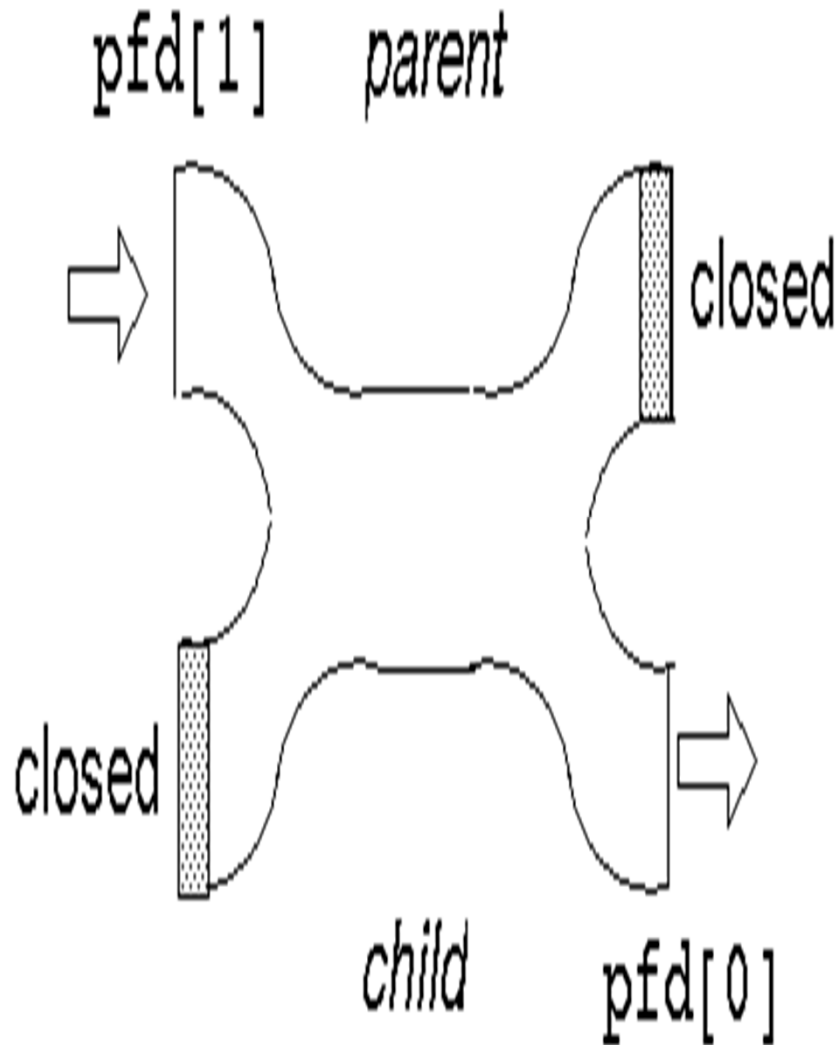
After
fork



This gives two read ends
and two write ends.

Either process can write into the
pipe, and either can read from it.

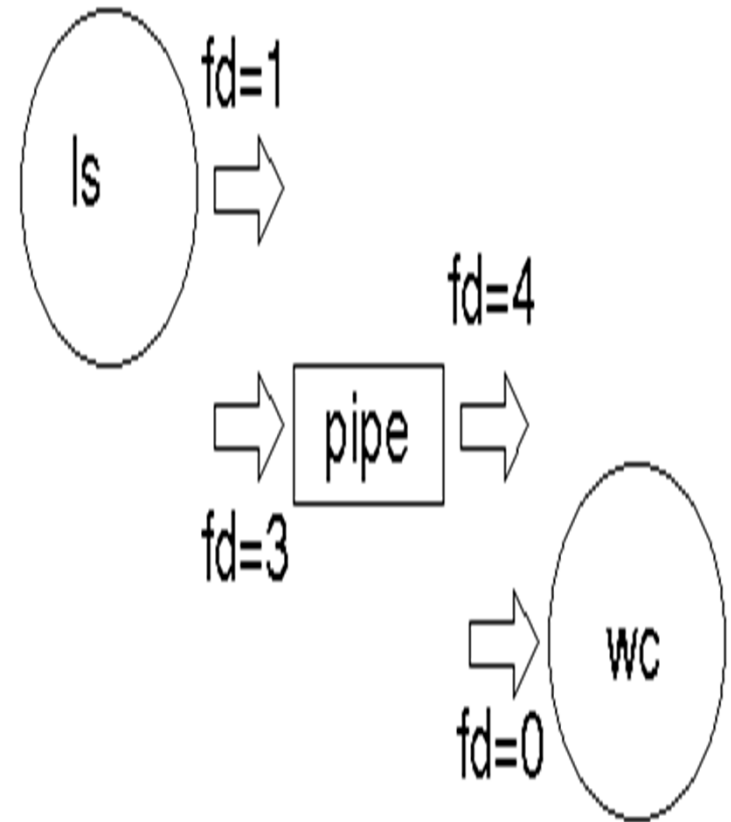
Fork and a pipe (Contd.)



- To ensure consistent and expected behavior, **one of the processes must close its read end, while the other process must close its write end.**
- Subsequently, it will revert to being a straightforward pipeline.
- Let's say the parent is the sender and the child is the receiver.
- The parent closes its read end.
- The child closed its write end.
- **pipe.c**

Example of pipe

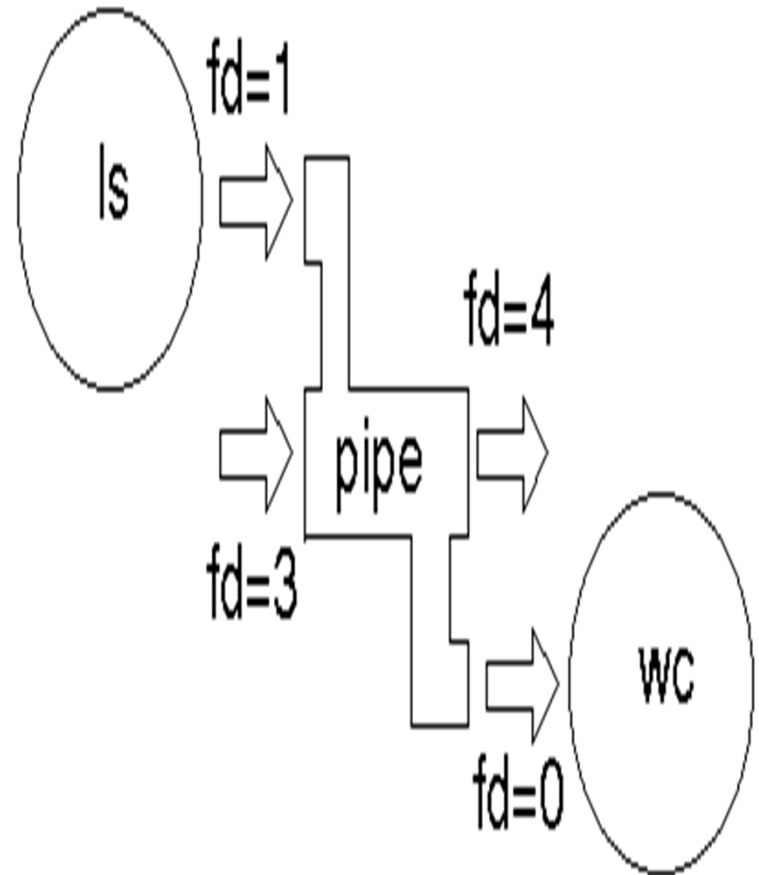
- To implement `ls | wc` the shell will have created a pipe and then forked.
- The parent will exec to be replaced by `ls`, and the child will exec to be replaced by `wc`
- The write end of the pipe may be descriptor 3 and the read end may be descriptor 4.
- `ls` normally writes to 1 and `wc` normally reads from 0.
- How do these get matched up?



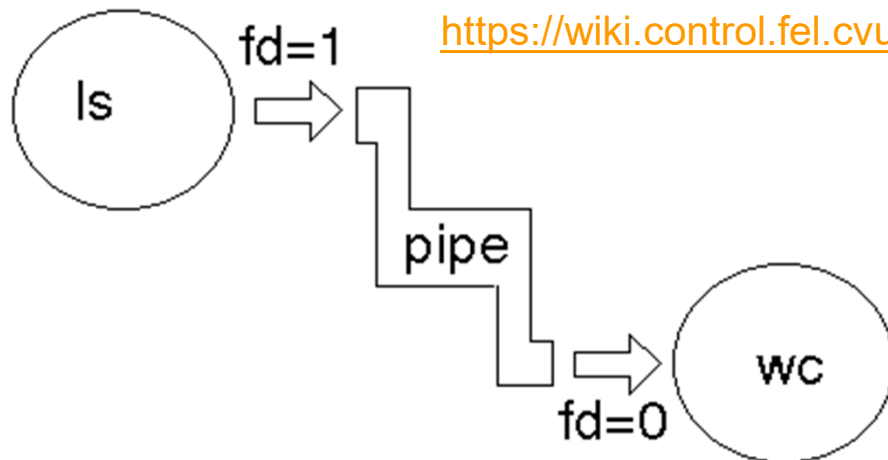
dup2 function

- The "dup2" function takes an existing file descriptor, and another one that it "would like to be".
- Here, fd=3 would also like to be 1, and fd=4 would like to be 0.
- So we dup2 fd=3 as 1, and dup2 fd=4 as 0.
- Then the old fd=3 and fd=4 can be closed as they are no longer needed.
- **dup2() is useful for redirecting file descriptors, such as redirecting standard output (stdout) to a file.**

Before
Close



After close



https://wiki.control.fel.cvut.cz/pos/os_api/unix-8.html

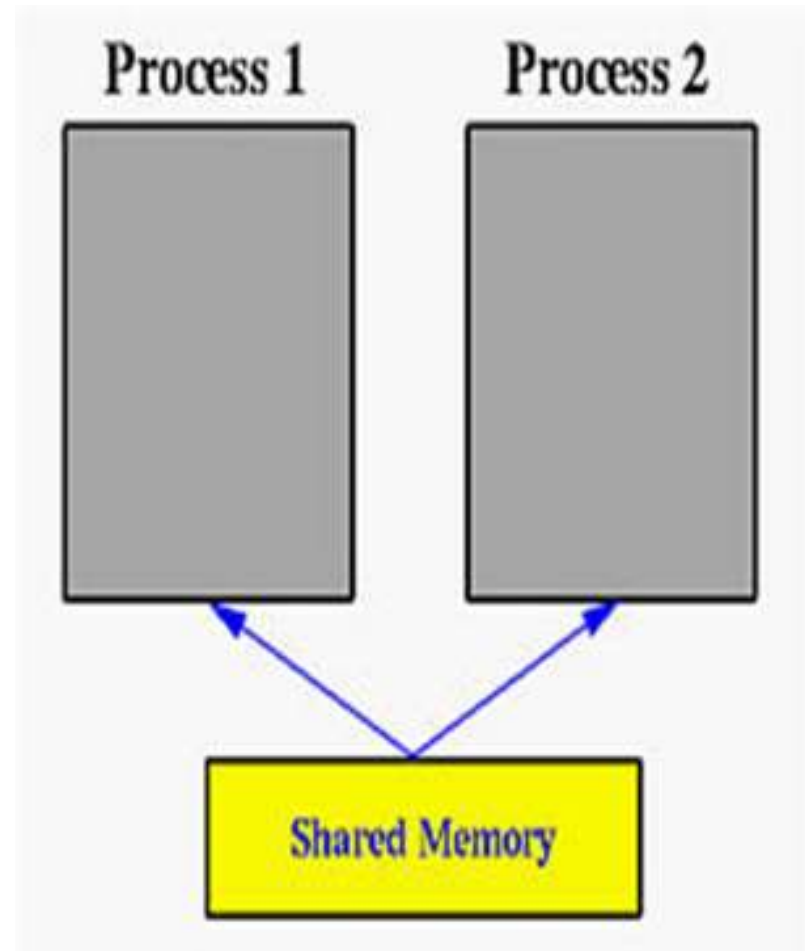
dup2.c

read/write from/to Pipe

- A process may want to both write to and read from a child.
- In this case it creates two pipes.
- One of these is used by the parent for writing and by the child for reading.
- The other is used by the child for writing and the parent for reading.
- `twopipe.c`

What is Shared Memory?

- The figure shows two processes and their address spaces.
- The yellow rectangle is a shared memory attached to both address spaces and both process 1 and process 2 can have access to this shared memory as if the shared memory is part of its own address space.
- Note that, **race conditions** may occur if memory accesses are not handled properly.



What is Shared Memory? (Contd.)

- Shared memory is a feature supported by UNIX-like operating systems.
- One process must explicitly ask for an area, using a key, to be shared by other processes. This process will be called the server.
- All other processes, the clients, that know the shared area can access it.
- However, there is no protection to a shared memory and any process that knows it can access it freely.
- To protect a shared memory from being accessed at the same time by several processes, a synchronization protocol must be setup.

A general scheme of using shared memory (Server)

- For a server, it should be started before any client. The server should perform the following tasks:
 - Ask for a shared memory with a memory key and memorize the returned shared memory ID. This is performed by system call `shmget()`.
 - Attach this shared memory to the server's address space with system call `shmat()`.
 - Initialize the shared memory, if necessary.
 - Do something and wait for all clients' completion.
 - Detach the shared memory with system call `shmdt()`.
 - Remove the shared memory with system call `shmctl()`.

A general scheme of using shared memory (client)

- For the client part, the procedure is almost the same:
 - Ask for a shared memory with the same memory key and memorize the returned shared memory ID.
 - Attach this shared memory to the client's address space.
 - Use the memory.
 - Detach all shared memory segments, if necessary.
 - Exit.

Programming Shared Memory

- A shared memory segment is identified by a unique integer, the shared memory ID.
- The program should start with the following lines:
 - `#include <sys/types.h>`
 - `#include <sys/ipc.h>`
 - `#include <sys/shm.h>`

Key

- Unix requires a key of type `key_t` defined in file `sys/types.h` for requesting resources such as shared memory segments, message queues and semaphores.
- A key is simply an integer of type `key_t`; however, you should not use `int` or `long`, since the length of a key is system dependent.
- There are three different ways of using keys, namely:
 - a specific integer value (e.g., 123456)
 - a key generated with function `ftok()`
 - a uniquely generated key using `IPC_PRIVATE` (i.e., a private key).

Key (Contd.)

- The first way is the easiest one; however, its use may be very risky since a process can access your resource as long as it uses the same key value to request that resource.
- The following example assigns 1234 to a key:
- `key_t SomeKey;`
- `SomeKey = 1234;`

ftok()

- The ftok() function has the following prototype:
- `key_t ftok( const char *path, /* a path string */`
- `int id /* an integer value */);`
- Function ftok() takes a character string that identifies a path and an integer (usually a character) value, and generates an integer of type key_t based on the first argument with the value of id in the most significant position.
- Thus, as long as processes use the same arguments to call ftok(), the returned key value will always be the same.
- The most commonly used value for the first argument is ".", the current directory.

ftok() (Contd.)

- If all related processes are stored in the same directory, the following call to ftok() will generate the same key value:
 - `#include <sys/types.h>`
 - `#include <sys/ipc.h>`
 - `key_t SomeKey;`
 - `SomeKey = ftok(".", 'x');`

IPC_PRIVATE

- In this case, the system will generate a **unique key and guarantee that no other process will have the same key.**
- If a resource is requested with IPC_PRIVATE in a place where a key is required, that process will receive a unique key for that resource.
- Since that resource is identified with a unique key unknown to the outsiders, other processes will not be able to share that resource and, as a result, the requesting process is guaranteed that it owns and accesses that resource exclusively.
- **IPC_PRIVATE is normally shared between related process such as parent/child, child/child.**

Asking for a Shared Memory Segment - shmget()

- It is defined as follows:
- `shm_id = shmget( key_t k, /* the key for the segment */`
- `int size, /* the size of the segment */`
- `int flag); /* create/use flag */`
- In the above definition, k is of type key_t or IPC_PRIVATE. size is the size of the requested shared memory.
- The purpose of flag is to specify the way that the shared memory will be used. For our purpose, only the following two values are important:
- `IPC_CREAT | 0666` for a server (i.e., creating and granting read and write access to the server)
- `0666` for any client (i.e., granting read and write access to the client)
- If shmget() can successfully get the shared memory, its function value is a non-negative integer, the shared memory ID; otherwise, the function value is negative.

Example of using shmget()

- `#include <sys/types.h>`
- `#include <sys/ipc.h>`
- `#include <sys/shm.h>`
- `#include <stdio.h>`
- `int shm_id; /* shared memory ID */`
- `shm_id = shmget(IPC_PRIVATE, 4*sizeof(int), IPC_CREAT | 0666);`
- `if (shm_id < 0) {`
- `printf("shmget error\n");`
- `exit(1);`
- `}`
- If a client wants to use a shared memory created with `IPC_PRIVATE`, it must be a child process of the server, **created after the parent has obtained the shared memory**, so that the private key value can be passed to the child when it is created.

Attaching a Shared Memory Segment to an Address Space - `shmat()`

- After a shared memory ID is returned, the next step is to attach it to the address space of a process.
- This is done with system call `shmat()`. The use of `shmat()` is as follows:
- `shm_ptr = shmat(int shmid, /* shared memory ID */`
- `const void *shmaddr, /* attached address, always NULL*/`
- `int flag); /* access flag, always 0 */`

shmat() (Contd.)

- System call shmat() accepts a shared memory ID, shm_id, and attaches the indicated shared memory to the program's address space.
- The returned value is a pointer of type (void *) to the attached shared memory. Thus, casting is usually necessary.
- If this call is unsuccessful, the return value is -1.
- Normally, the second parameter is NULL. Let the OS desire the attached memory address.
- If the flag is SHM_RDONLY, this shared memory is attached as a read-only memory; Typically, this is set to 0 for default behavior.

Detaching a Shared Memory Segment - `shmdt()`

- System call `shmdt()` is used to detach a shared memory.
- After a shared memory is detached, it cannot be used.
- However, **it is still there and can be re-attached back to a process's address space, perhaps at a different address.**
- To remove a shared memory, use **`shmctl()`**, discuss next.
- The only argument of the call to `shmdt()` is the shared memory address returned by `shmat()`.

Thus, the following code detaches the shared memory from a program:

- **`shmdt(shm_ptr);`**
- where `shm_ptr` is the pointer to the shared memory.
- This pointer is returned by `shmat()` when the shared memory is attached.
- If the detach operation fails, the returned function value is non-zero.

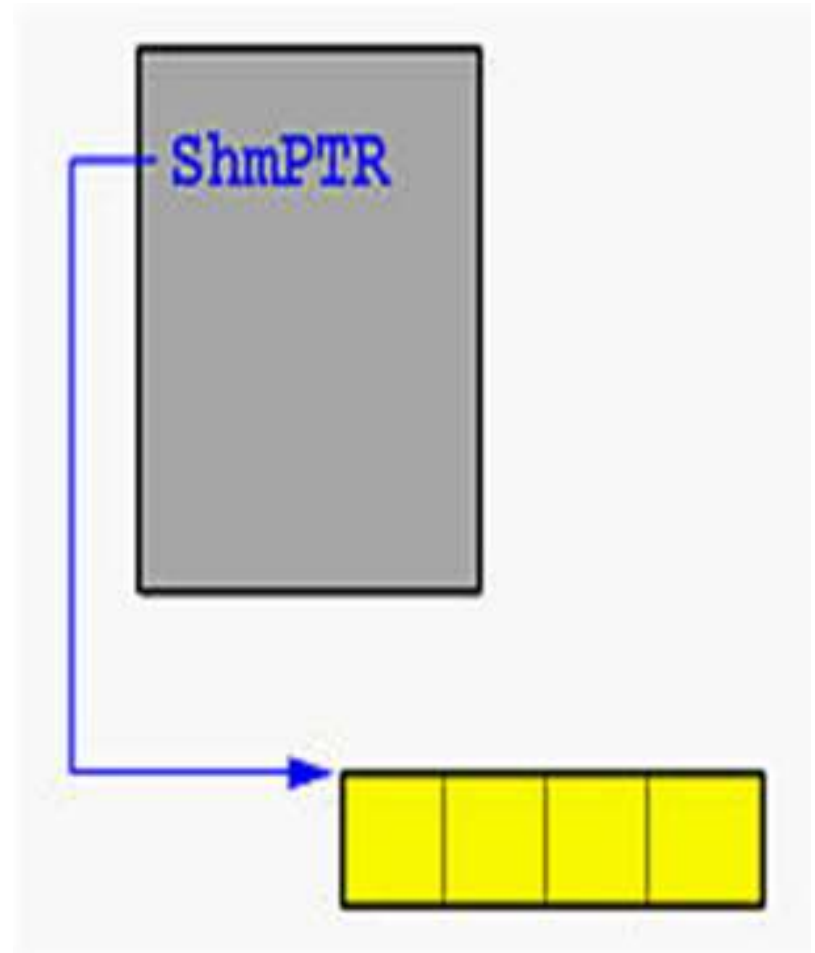
Removing a Shared Memory Segment – shmctl()

- To remove a shared memory segment, use the following code:
- `shmctl(shm_id, IPC_RMID, NULL);`
- where `shm_id` is the shared memory ID. `IPC_RMID` indicates this is a remove operation.
- Note that after the removal of a shared memory segment, if you want to use it again, you use `shmget()` followed by `shmat()`.

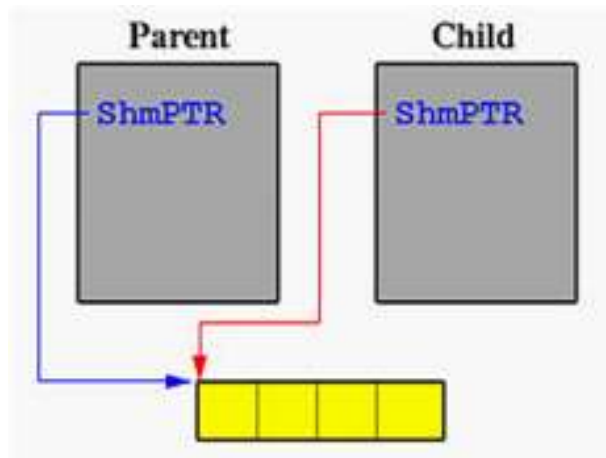
Parent and Child Share Memory

- see `shm_parent_child.c`

Before
fork



After
fork



Client-Server Shared Memory

- see `server.c`, `client.c`
- Note that the above code assumes the server and client programs are in the current directory.
- For the client to run correctly, the server must be started first and the client can only be started after the server has successfully obtained the shared memory.

Operating Systems

No. 6

Sarun Intakosum

Process Synchronization

Producer

```
while (true) {  
  
    /* produce an item and put in nextProduced */  
    while (count == BUFFER_SIZE)  
        ; // do nothing  
    buffer [in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}
```

Consumer

```
while (true) {  
    while (count == 0)  
        ; // do nothing  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item in nextConsumed  
    */  
}
```

Race Condition

- `count++` could be implemented as

```
register1 = count  
register1 = register1 + 1  
count = register1
```

- `count--` could be implemented as

```
register2 = count  
register2 = register2 - 1  
count = register2
```

- Consider this execution interleaving with “count = 5” initially:

```
S0: producer execute register1 = count {register1 = 5}  
S1: producer execute register1 = register1 + 1 {register1 = 6}  
S2: consumer execute register2 = count {register2 = 5}  
S3: consumer execute register2 = register2 - 1 {register2 = 4}  
S4: producer execute count = register1 {count = 6}  
S5: consumer execute count = register2 {count = 4}
```

Process Synchronization Software

- For a successful process synchronization:
 - Used resource must be locked from other processes until released
 - A waiting process is allowed to use the resource only when it is released
- A mistake could leave deadlock.

Process Synchronization Software (continued)

- **Critical region(section) :** A part of a program that must complete execution before other processes can have access to the resources being used

Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
 2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
 3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
- Assume that each process executes at a nonzero speed
 - No assumption concerning relative speed of the N processes

Peterson's Solution

- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted.
- The two processes share two variables:
 - int **turn**;
 - Boolean **flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section.
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i]** = true implies that process P_i is ready!

Algorithm for Process P_i

```
while (true) {  
    flag[i] = TRUE;  
    turn = j;  
    while ( flag[j] && turn == j);
```

CRITICAL SECTION

```
    flag[i] = FALSE;
```

REMAINDER SECTION

```
}
```

Bakery Algorithm

Critical section for n processes

- Before entering its critical section, process receives a number. Holder of the smallest number enters the critical section.
- If processes P_i and P_j receive the same number, if $i < j$, then P_i is served first; else P_j is served first.
- The numbering scheme always generates numbers in increasing order of enumeration; i.e., 1,2,3,3,3,3,4,5...

Bakery Algorithm

- Notation $<\equiv$ lexicographical order (ticket #, process id #)
 - $(a,b) < (c,d)$ if $a < c$ or if $a = c$ and $b < d$
 - $\max(a_0, \dots, a_{n-1})$ is a number, k , such that $k \geq a_i$ for $i = 0, \dots, n-1$
- Shared data
 - `boolean choosing[n];`
 - `int number[n];`

Data structures are initialized to **false** and **0** respectively

Bakery Algorithm

```
do {  
    choosing[i] = true;  
    number[i] = max(number[0], number[1], ..., number [n - 1])+1;  
    choosing[i] = false;  
    for (j = 0; j < n; j++) {  
        while (choosing[j]) ;  
        while ((number[j] != 0) && ((number[j],j) < (number[i],i)) ;  
    }  
        critical section  
    number[i] = 0;  
        remainder section  
} while (1);
```

Semaphore

- Synchronization tool that does not require busy waiting
- Semaphore S – integer variable
- Two standard operations modify S : `wait()` and `signal()`
 - Originally called `P()` and `V()`
- Less complicated
- Can only be accessed via two indivisible (atomic) operations
 - `wait (S) {`
 - `while S <= 0`
 - `; // no-op`
 - `S--;`
 - `}`
 - `signal (S) {`
 - `S++;`
 - `}`

Semaphore as General Synchronization Tool

- **Counting** semaphore – integer value can range over an unrestricted domain
- **Binary** semaphore – integer value can range only between 0 and 1; can be simpler to implement
 - Also known as **mutex locks**
- Can implement a counting semaphore **S** as a binary semaphore
- Provides mutual exclusion
 - Semaphore S; // initialized to 1
 - wait (S);
 Critical Section
 signal (S);

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue. Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list
- Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue.
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue.

Semaphore Implementation with no Busy waiting (Cont.)

- Implementation of wait:

```
wait (S){  
    value--;  
    if (value < 0) {  
        add this process to waiting queue  
        block(); }  
}
```

- Implementation of signal:

```
Signal (S){  
    value++;  
    if (value <= 0) {  
        remove a process P from the waiting queue  
        wakeup(P); }  
}
```

Classical Problems of Synchronization

- Bounded-Buffer Problem
- Readers and Writers Problem
- Dining-Philosophers Problem

Bounded-Buffer Problem

- N buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value N .

Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
while (true) {  
  
    // produce an item  
  
    wait (empty);  
    wait (mutex);  
  
    // add the item to the buffer  
  
    signal (mutex);  
    signal (full);  
}
```

Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
while (true) {  
    wait (full);  
    wait (mutex);  
  
    // remove an item from buffer  
  
    signal (mutex);  
    signal (empty);  
  
    // consume the removed item  
  
}
```

Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - Readers – only read the data set; they do **not** perform any updates
 - Writers – can both read and write.
- Problem – allow multiple readers to read at the same time. Only one single writer can access the shared data at the same time.
- Shared Data
 - Data set
 - Semaphore **mutex** initialized to 1.
 - Semaphore **wrt** initialized to 1.
 - Integer **readcount** initialized to 0.

Readers-Writers Problem (Cont.)

- The structure of a writer process

```
while (true) {  
    wait (wrt) ;  
  
    //  writing is performed  
  
    signal (wrt) ;  
}
```

Readers-Writers Problem (Cont.)

- The structure of a reader process

```
while (true) {  
    wait (mutex) ;  
    readcount ++ ;  
    if (readcount == 1) wait (wrt) ;  
    signal (mutex)  
  
    // reading is performed  
  
    wait (mutex) ;  
    readcount -- ;  
    if (readcount == 0) signal (wrt) ;  
    signal (mutex) ;  
}
```

Dining-Philosophers Problem



- Shared data
 - Bowl of rice (data set)
 - Semaphore **chopstick** [5] initialized to 1

Dining-Philosophers Problem (Cont.)

- The structure of Philosopher i :

```
While (true) {  
    wait ( chopstick[i] );  
    wait ( chopStick[ (i + 1) % 5] );  
  
    // eat  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
  
    // think  
  
}
```

Problems with Semaphores

- Correct use of semaphore operations:
 - signal (mutex) wait (mutex)
 - wait (mutex) ... wait (mutex)
 - Omitting of wait (mutex) or signal (mutex) (or both)

Monitors

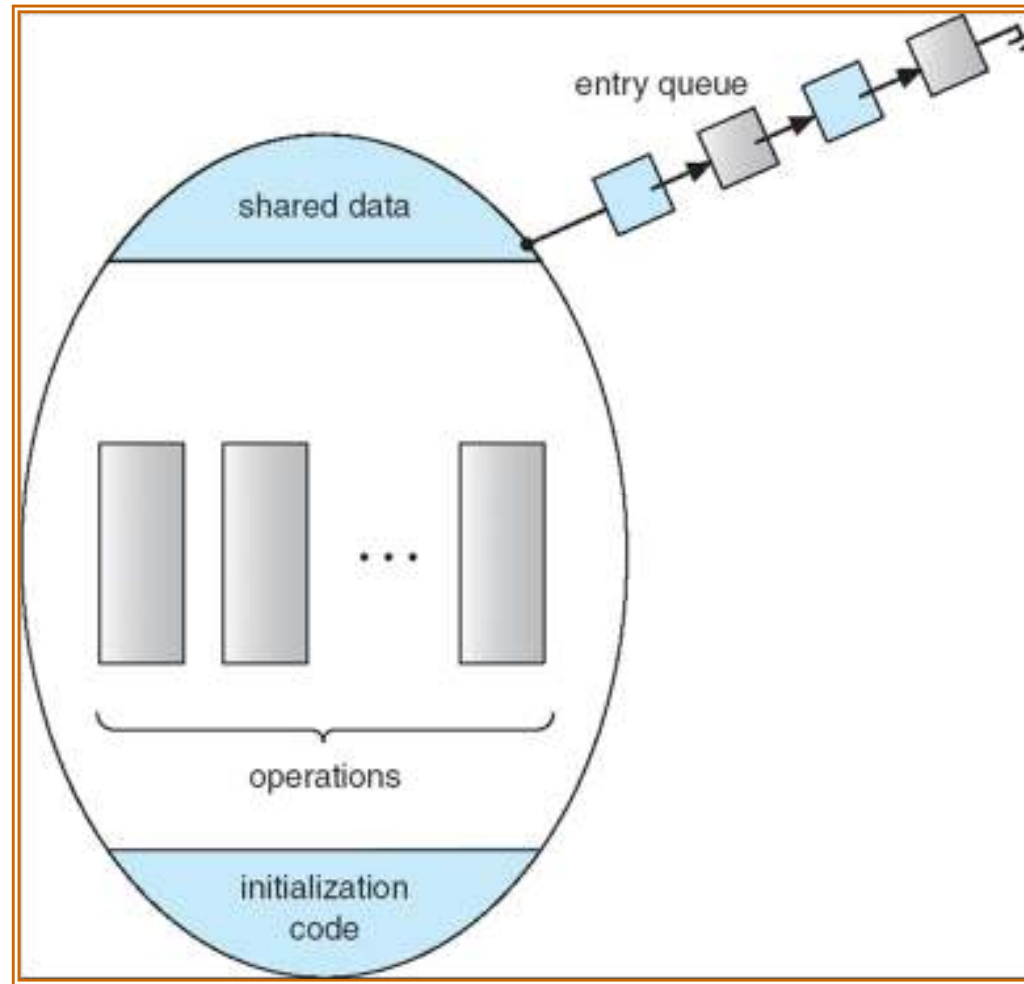
- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Only one process may be active within the monitor at a time

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }
    ...

    procedure Pn (...) {.....}

    Initialization code ( ....) { ... }
    ...
}
}
```

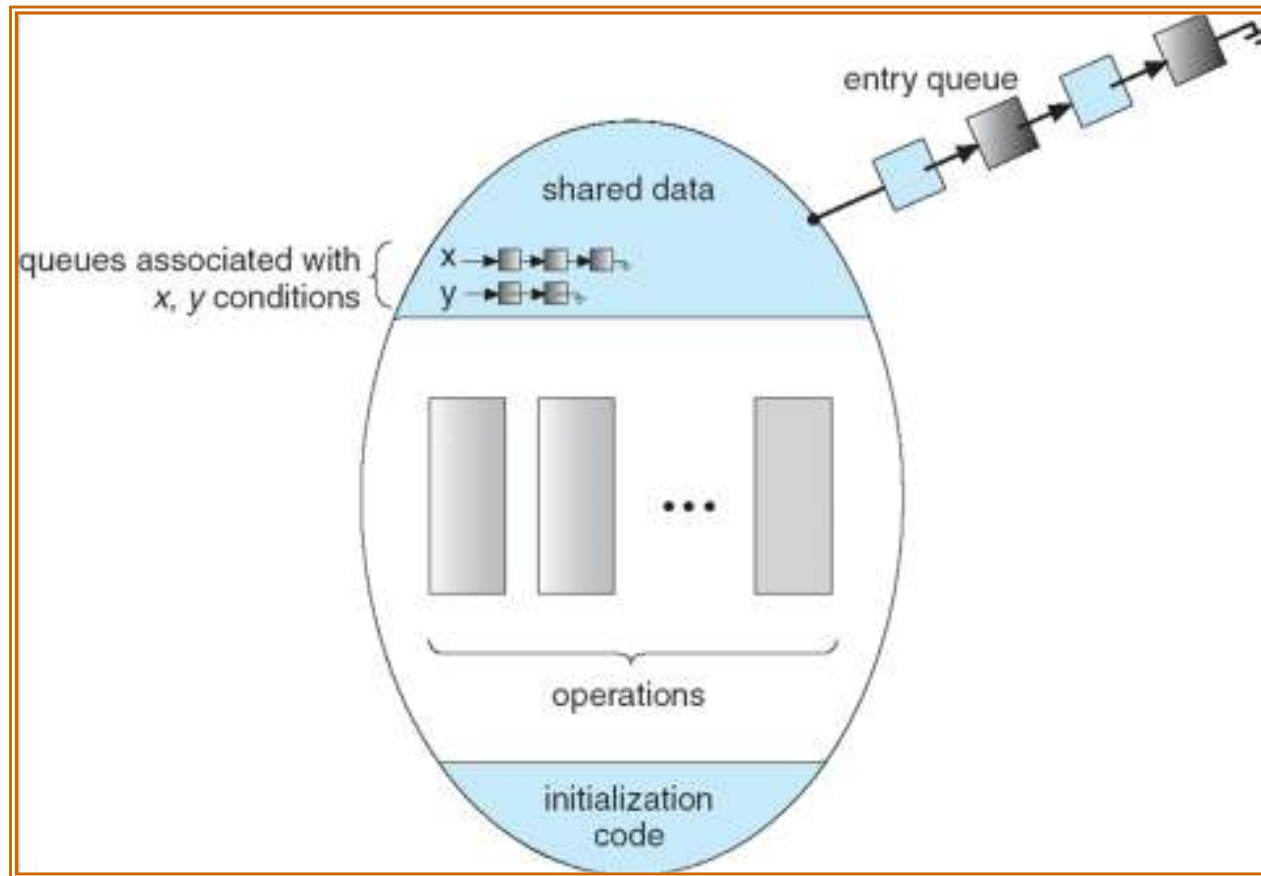
Schematic view of a Monitor



Condition Variables

- `condition x, y;`
- Two operations on a condition variable:
 - `x.wait ()` – a process that invokes the operation is suspended.
 - `x.signal ()` – resumes one of processes (if any) that invoked `x.wait ()`

Monitor with Condition Variables



Solution to Dining Philosophers

monitor DP

```
{  
    enum { THINKING; HUNGRY, EATING) state [5] ;  
    condition self [5];  
  
    void pickup (int i) {  
        state[i] = HUNGRY;  
        test(i);  
        if (state[i] != EATING) self [i].wait;  
    }  
  
    void putdown (int i) {  
        state[i] = THINKING;  
        // test left and right neighbors  
        test((i + 4) % 5);  
        test((i + 1) % 5);  
    }  
}
```

Solution to Dining Philosophers (cont)

```
void test (int i) {  
    if ( (state[(i + 4) % 5] != EATING) &&  
        (state[i] == HUNGRY) &&  
        (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}
```

```
    initialization_code() {  
        for (int i = 0; i < 5; i++)  
            state[i] = THINKING;  
    }  
}
```

Solution to Dining Philosophers (cont)

- Each philosopher i invokes the operations `pickup()` and `putdown()` in the following sequence:

`dp.pickup (i)`

EAT

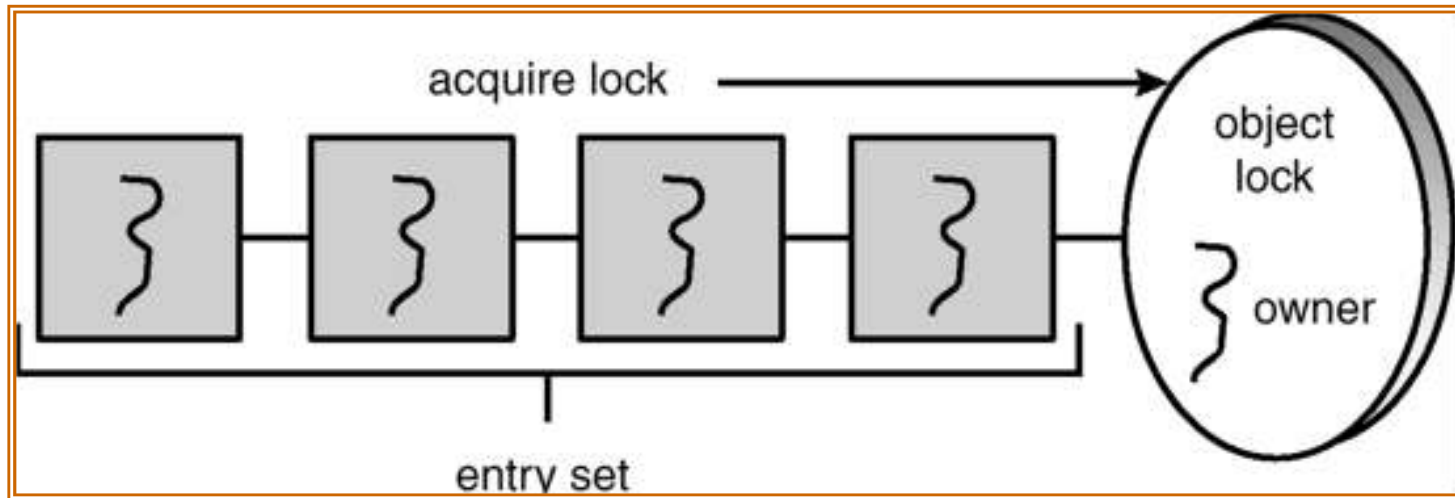
`dp.putdown (i)`

Java Synchronization

synchronized Statement

- Every object has a lock associated with it
- Calling a synchronized method requires “owning” the lock
- If a calling thread does not own the lock (another thread already owns it), the calling thread is placed in the wait set for the object’s lock
- The lock is released when a thread exits the synchronized method

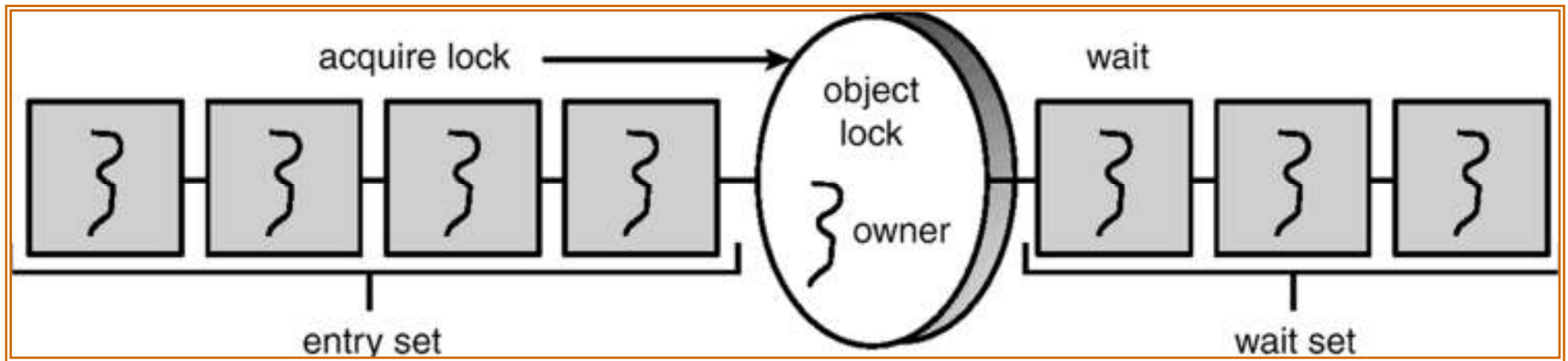
Entry Set



The wait() Method

- When a thread calls `wait()`, the following occurs:
 1. the thread releases the object lock
 2. thread state is set to blocked
 3. thread is placed in the wait set

Entry and Wait Sets



The notify() Method

When a thread calls `notify()`, the following occurs:

1. selects an arbitrary thread T from the wait set
2. moves T to the entry set
3. sets T to Runnable

T can now compete for the object's lock again

insert() with wait/notify Methods

```
public synchronized void insert(Object item) {  
    while (count == BUFFER SIZE) {  
        try {  
            wait();  
        }  
        catch (InterruptedException e) { }  
    }  
    ++count;  
    buffer[in] = item;  
    in = (in + 1) % BUFFER SIZE;  
    notify();  
}
```

remove() with wait/notify Methods

```
public synchronized Object remove() {  
    Object item;  
    while (count == 0) {  
        try {  
            wait();  
        }  
        catch (InterruptedException e) { }  
    }  
    --count;  
    item = buffer[out];  
    out = (out + 1) % BUFFER SIZE;  
    notify();  
    return item;  
}
```

Multiple Notifications

- `notify()` selects an arbitrary thread from the wait set.
*This may not be the thread that you want to be selected.
- Java does not allow you to specify the thread to be selected
- `notifyAll()` removes ALL threads from the wait set and places them in the entry set. This allows the threads to decide among themselves who should proceed next.
- `notifyAll()` is a conservative strategy that works best when multiple threads may be in the wait set

Java Semaphores

- Java before Java 5 does not provide a semaphore, but a basic semaphore can be constructed using Java synchronization mechanism

Semaphore Class

```
public class Semaphore
{
    private int value;
    public Semaphore() {
        value = 0;
    }
    public Semaphore(int value) {
        this.value = value;
    }
}
```

Semaphore Class (cont)

```
public synchronized void acquire() {  
    while (value == 0)  
        try {  
            wait();  
        } catch (InterruptedException ie) { }  
    value--;  
}  
public synchronized void release() {  
    ++value;  
    notify();  
}  
}
```

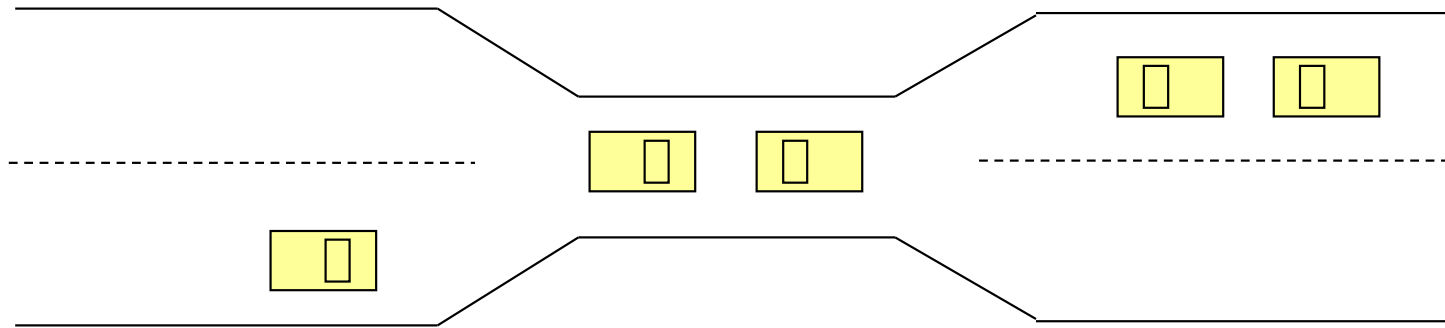
Operating Systems

No. 7

Sarun Intakosum

Deadlock Management

Bridge Crossing Example



- ❑ Traffic only in one direction
- ❑ Each section of a bridge can be viewed as a resource
- ❑ If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback)
- ❑ Several cars may have to be backed up if a deadlock occurs
- ❑ Starvation is possible
- ❑ Note – Most OSes do not prevent or deal with deadlocks

Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let **S** and **Q** be two semaphores initialized to 1

P_0
wait (S);
wait (Q);

.
.
.
signal (S);
signal (Q);

P_1
wait (Q);
wait (S);

.
.
.
signal (Q);
signal (S);

- **Starvation** – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended.

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .
- Removal of only one condition can resolve the deadlock

Modeling Deadlocks

- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - request
 - use
 - release

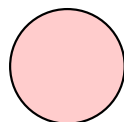
Resource-Allocation Graph

A set of vertices V and a set of edges E .

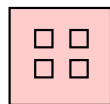
- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.
- request edge – directed edge $P_i \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph (Cont.)

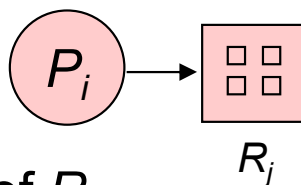
- Process



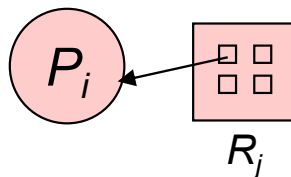
- Resource Type with 4 instances



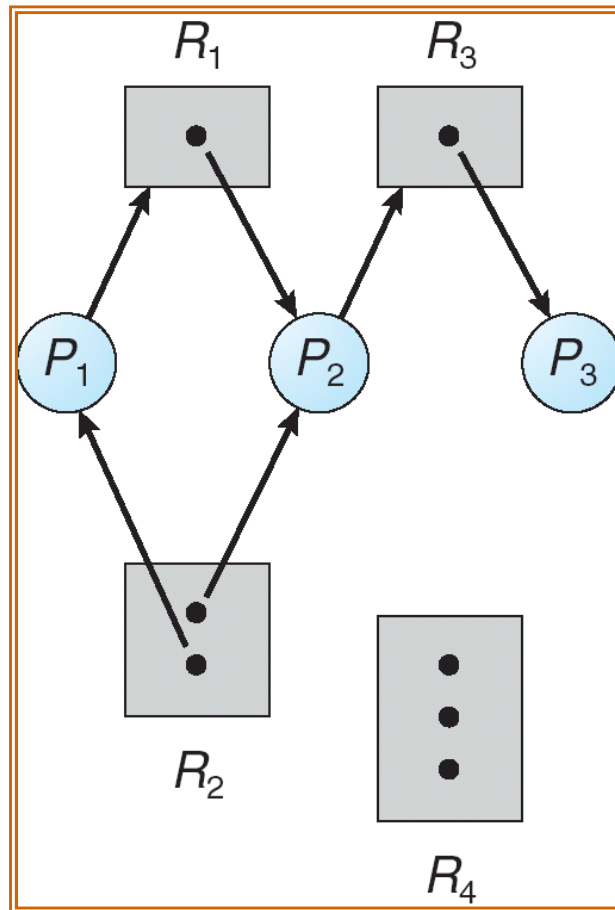
- P_i requests instance of R_j



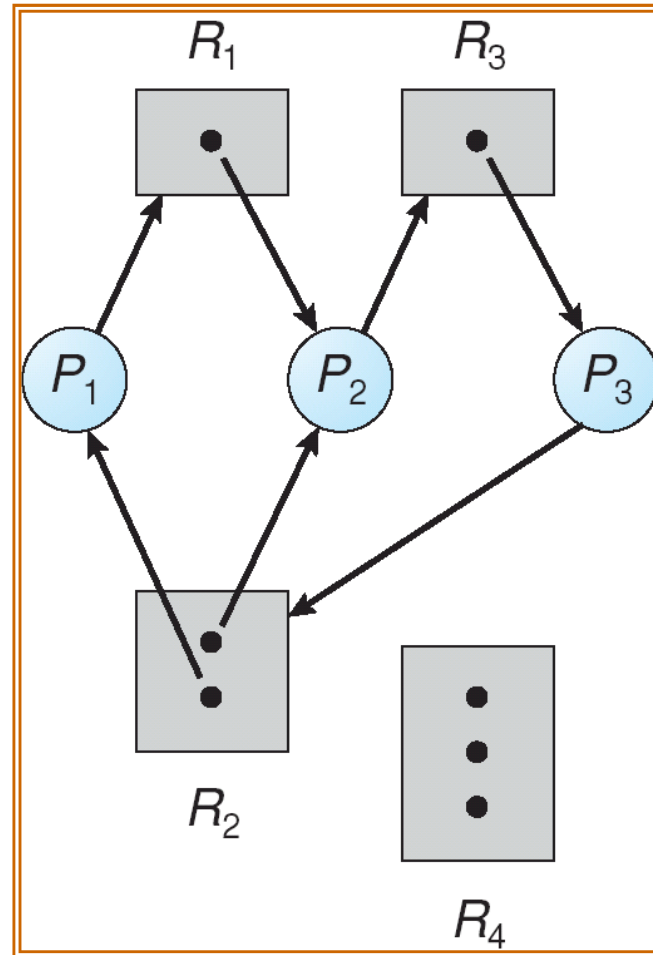
- P_i is holding an instance of R_j



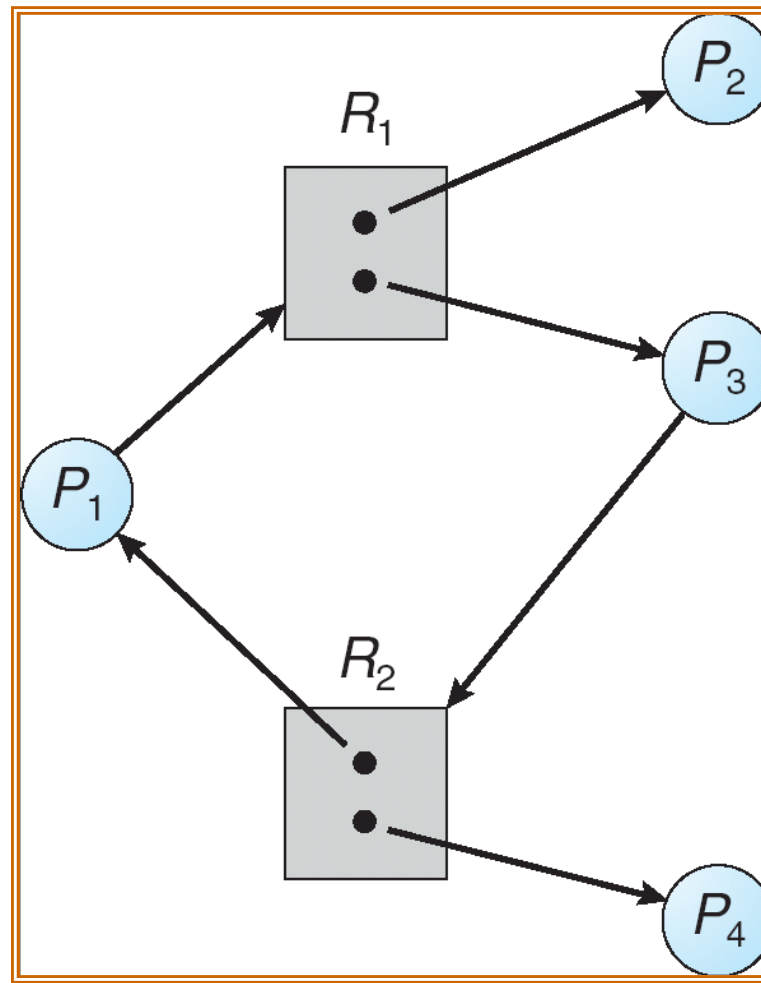
Example of a Resource Allocation Graph



Resource Allocation Graph With A Deadlock



Graph With A Cycle But No Deadlock



Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock.
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock.
 - if several instances per resource type, possibility of deadlock.

Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

Deadlock Prevention

Restrain the ways request can be made

- **Mutual Exclusion** – not required for sharable resources; must hold for nonsharable resources
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none
 - Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

- **No Preemption** –
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
 - Preempted resources are added to the list of resources for which the process is waiting
 - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration

Deadlock Avoidance

Requires that the system has some additional *a priori* information available.

- Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes.

Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks.
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock.
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

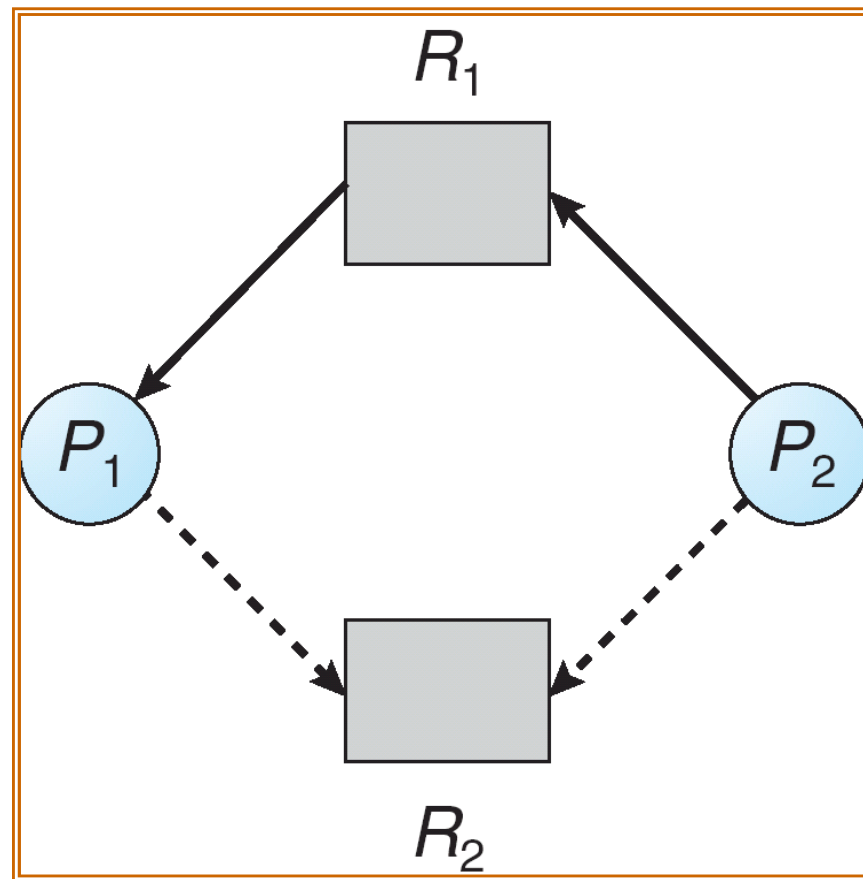
Avoidance algorithms

- Single instance of a resource type. Use a resource-allocation graph
- Multiple instances of a resource type. Use the banker's algorithm

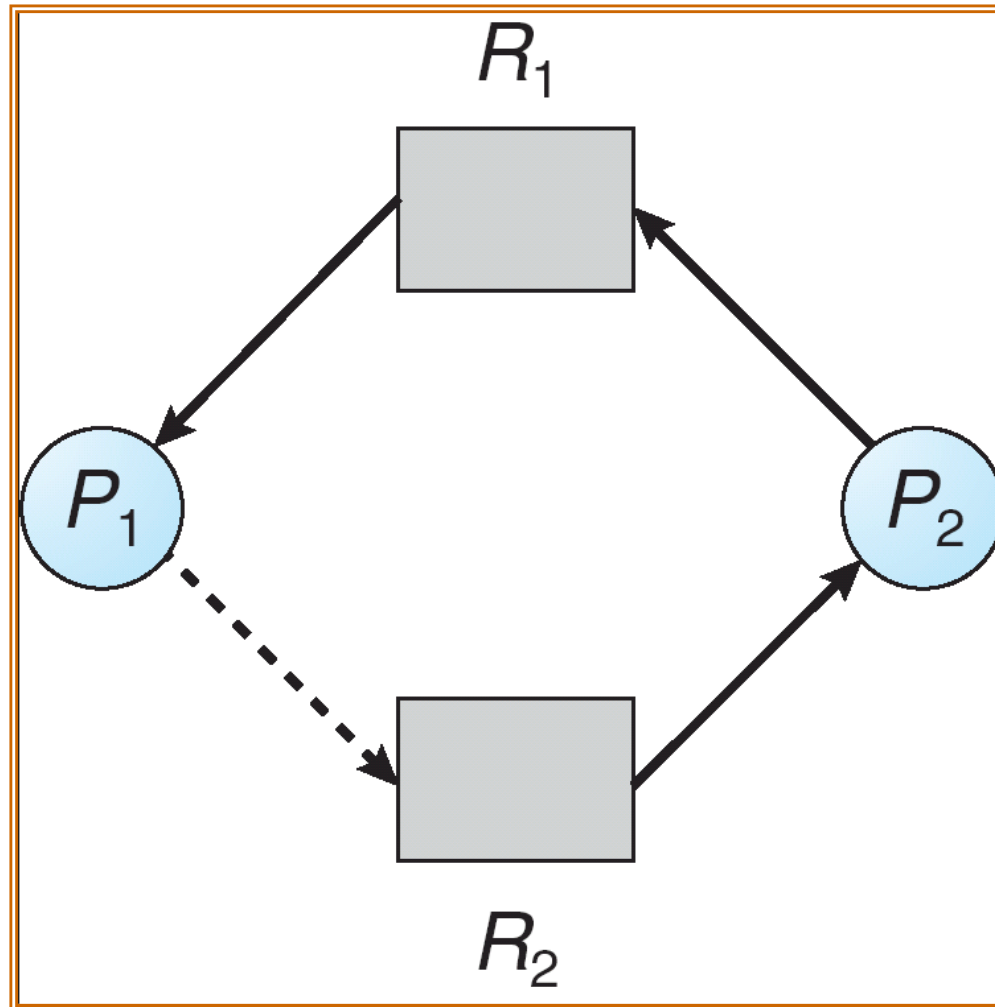
Resource-Allocation Graph Scheme

- *Claim edge* $P_i \rightarrow R_j$ indicated that process P_i may request resource R_j ; represented by a dashed line.
- Claim edge converts to request edge when a process requests a resource.
- Request edge converted to an assignment edge when the resource is allocated to the process.
- When a resource is released by a process, assignment edge reconverts to a claim edge.
- Resources must be claimed *a priori* in the system.

Resource-Allocation Graph



Unsafe State In Resource-Allocation Graph



Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

Banker's Algorithm for multiple resource types

Let n = number of processes, and m = number of resources types.

- **Available**: Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available.
- **Max**: $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j .
- **Allocation**: $n \times m$ matrix. If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j .
- **Need**: $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task.

$$Need[i,j] = Max[i,j] - Allocation[i,j].$$

Safety Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize:

Work = *Available*

Finish [i] = *false* for $i = 0, 1, \dots, n-1$.

2. Find and i such that both:

(a) *Finish* [i] = *false*

(b) $Need_i \leq Work$

If no such i exists, go to step 4.

3. $Work = Work + Allocation_i$
Finish [i] = *true*
go to step 2.

4. If *Finish* [i] == *true* for all i , then the system is in a safe state.

Resource-Request Algorithm for Process P_i

$Request$ = request vector for process P_i . If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j .

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available.
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to P_i .
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances).

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example (Cont.)

- The content of the matrix *Need* is defined to be *Max* – *Allocation*.

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria.

Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true.

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 1	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement.
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?

Banker's Algorithm (cont.)

❑ **Problems with Banker's Algorithm:**

- ❑ Jobs must state the maximum number of resources needed
- ❑ Number of total resources for each class must remain constant
- ❑ Number of jobs must remain fixed
- ❑ Overhead cost incurred can be quite high
- ❑ Resources aren't well utilized because the algorithm assumes the worst case
- ❑ Scheduling suffers as a result of poor utilization and jobs are kept waiting for resource allocation

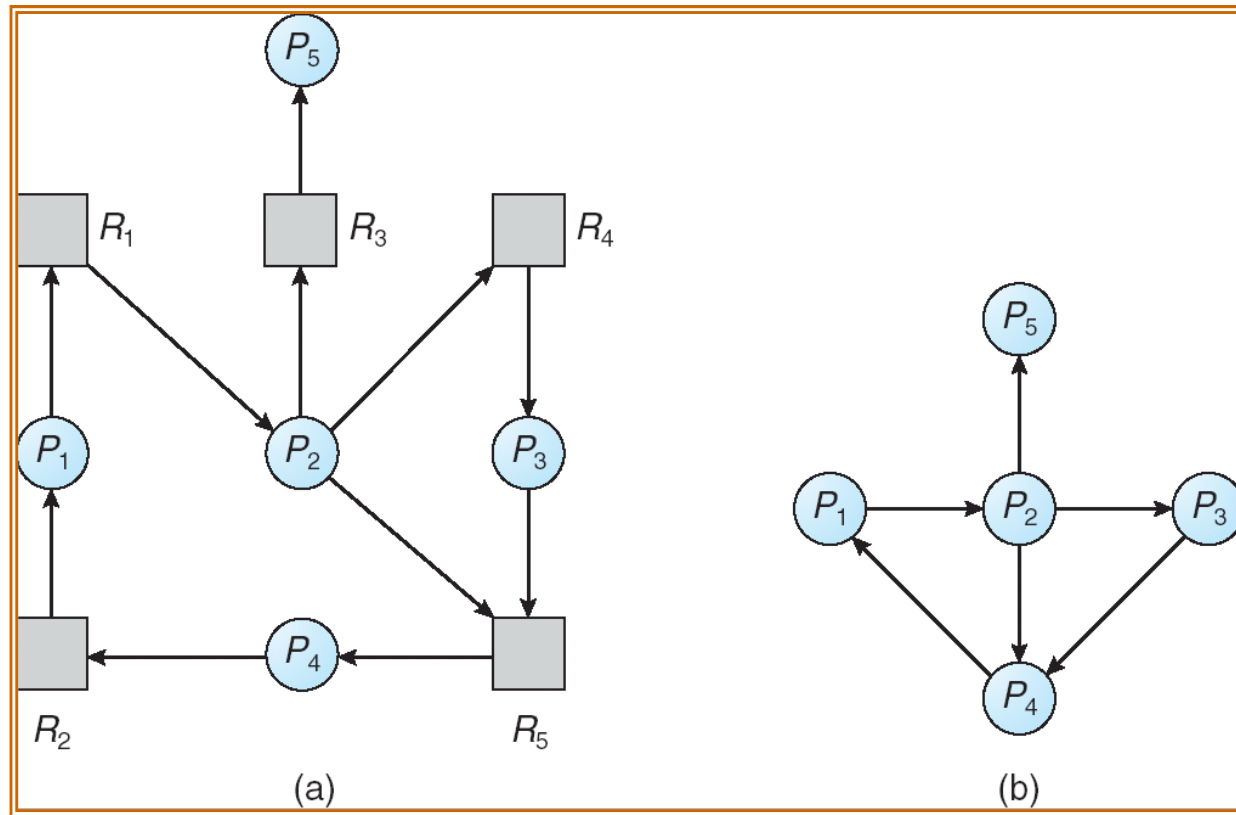
Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

Single Instance of Each Resource Type

- Maintain *wait-for* graph
 - Nodes are processes.
 - $P_i \rightarrow P_j$ if P_i is waiting for P_j .
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph

Corresponding wait-for graph

Several Instances of a Resource Type

- **Available:** A vector of length m indicates the number of available resources of each type.
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request:** An $n \times m$ matrix indicates the current request of each process. If $Request[i_j] = k$, then process P_i is requesting k more instances of resource type. R_j .

Detection Algorithm

1. Let *Work* and *Finish* be vectors of length m and n , respectively
Initialize:

(a) *Work* = *Available*

(b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then
 $Finish[i] = \text{false}$; otherwise, $Finish[i] = \text{true}$.

2. Find an index i such that both:

(a) $Finish[i] == \text{false}$

(b) $Request_i \leq Work$

If no such i exists, go to step 4.

Detection Algorithm (Cont.)

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2.
4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked.

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state.

Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances).
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = \text{true}$ for all i .

Example (Cont.)

- P_2 requests an additional instance of type C.

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	1
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests.
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4 .

Recovery from Deadlock: Process Termination

- ❑ Abort all deadlocked processes.
- ❑ Abort one process at a time until the deadlock cycle is eliminated.
- ❑ In which order should we choose to abort?
 - ❑ Priority of the process.
 - ❑ How long process has computed, and how much longer to completion.
 - ❑ Resources the process has used.
 - ❑ Resources process needs to complete.
 - ❑ How many processes will need to be terminated.
 - ❑ Is process interactive or batch?

Recovery from Deadlock: Resource Preemption

- ❑ Selecting a victim – minimize cost.
- ❑ Rollback – return to some safe state, restart process for that state.
- ❑ Starvation – same process may always be picked as victim, include number of rollback in cost factor.

Operating Systems

No. 8

Sarun Intakosum

Real Memory Management

Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., base and limit registers)

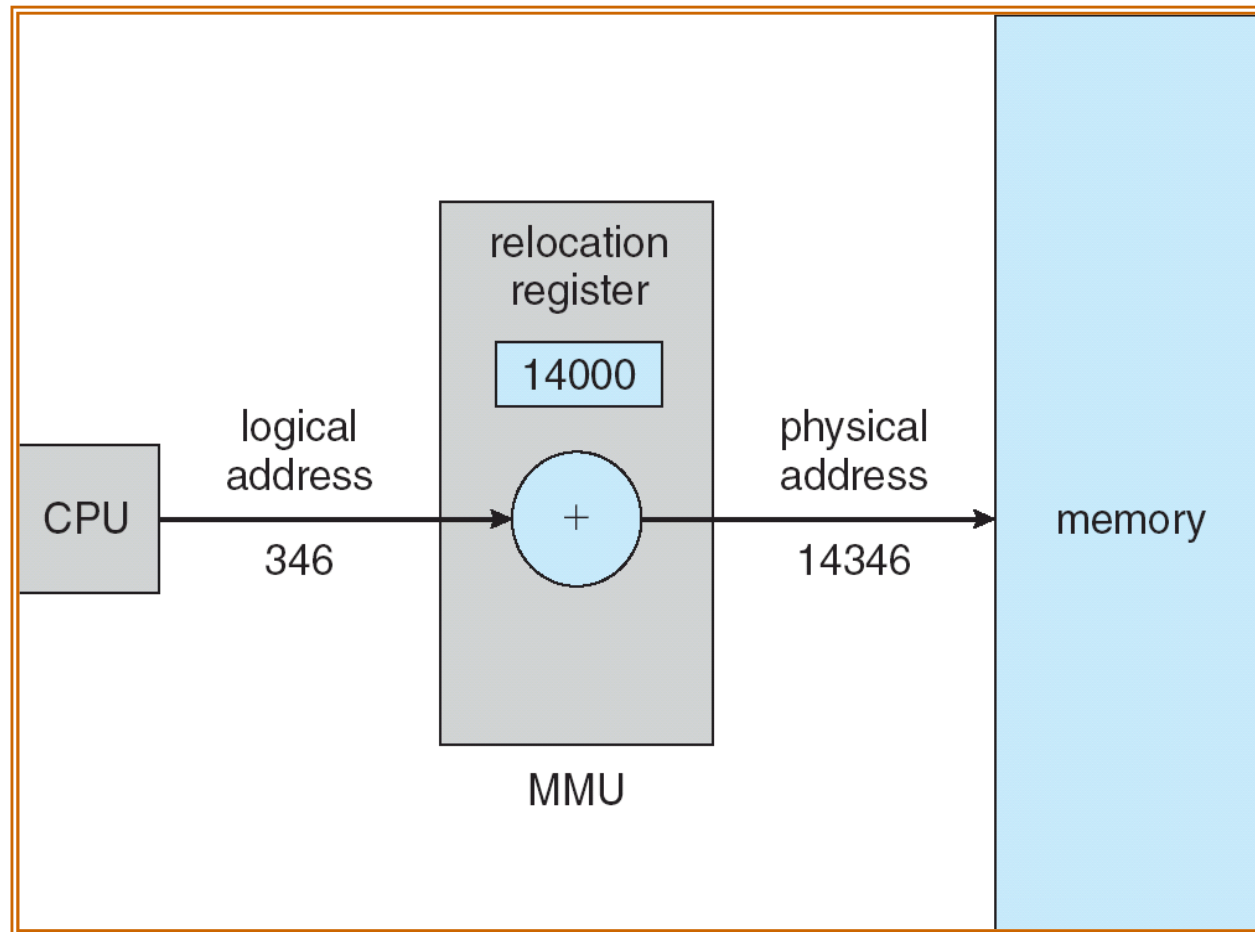
Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

Memory-Management Unit (MMU)

- ❑ Hardware device that maps virtual to physical address
- ❑ In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- ❑ The user program deals with *logical* addresses; it never sees the *real* physical addresses

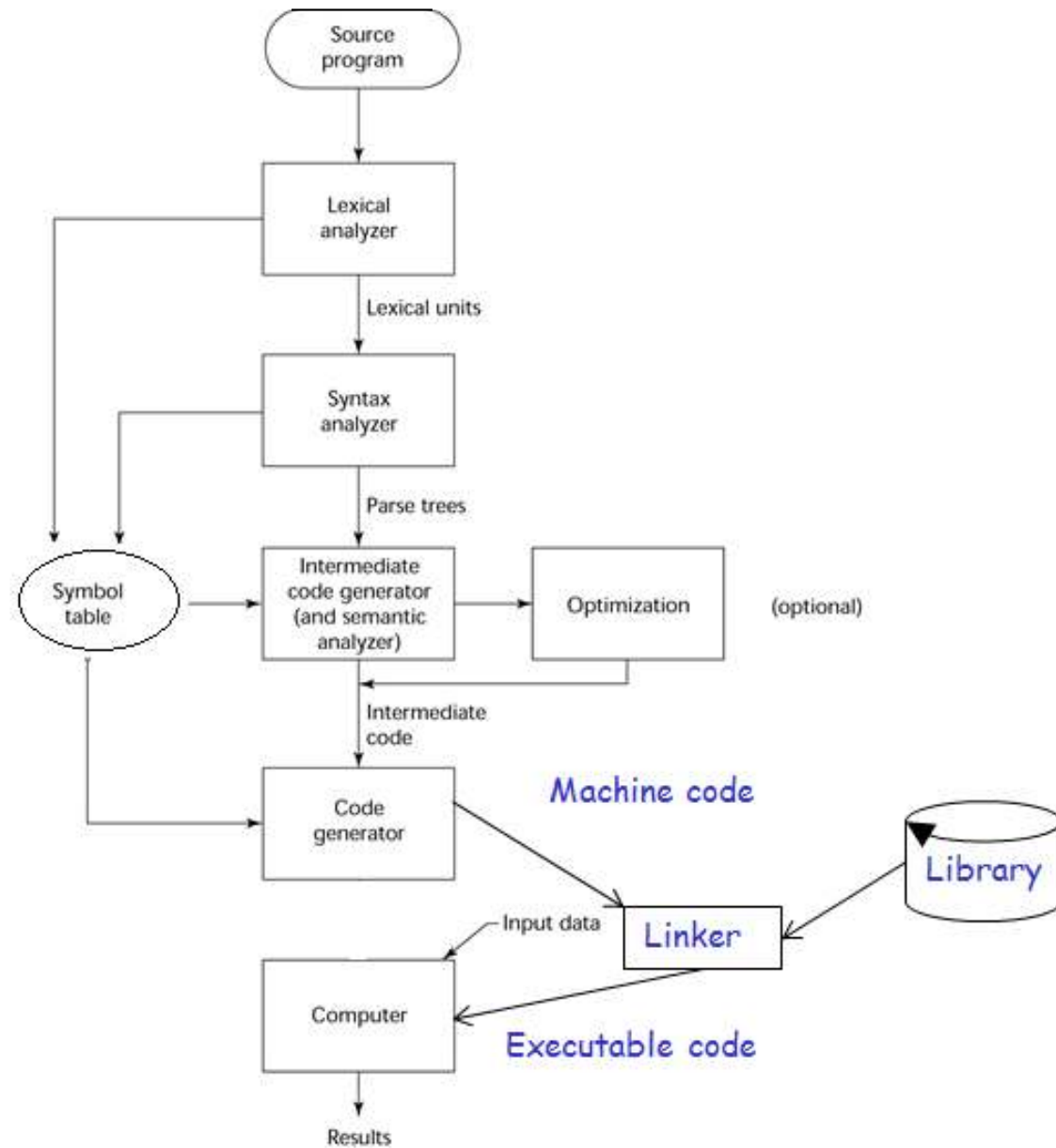
Dynamic relocation using a relocation register



Dynamic Linking

- ❑ Linking postponed until execution time
- ❑ Small piece of code, *stub*, used to locate the appropriate memory-resident library routine
- ❑ Stub replaces itself with the address of the routine, and executes the routine
- ❑ Operating system needed to check if routine is in processes' memory address
- ❑ Dynamic linking is particularly useful for libraries
- ❑ System also known as **shared libraries**

Linking

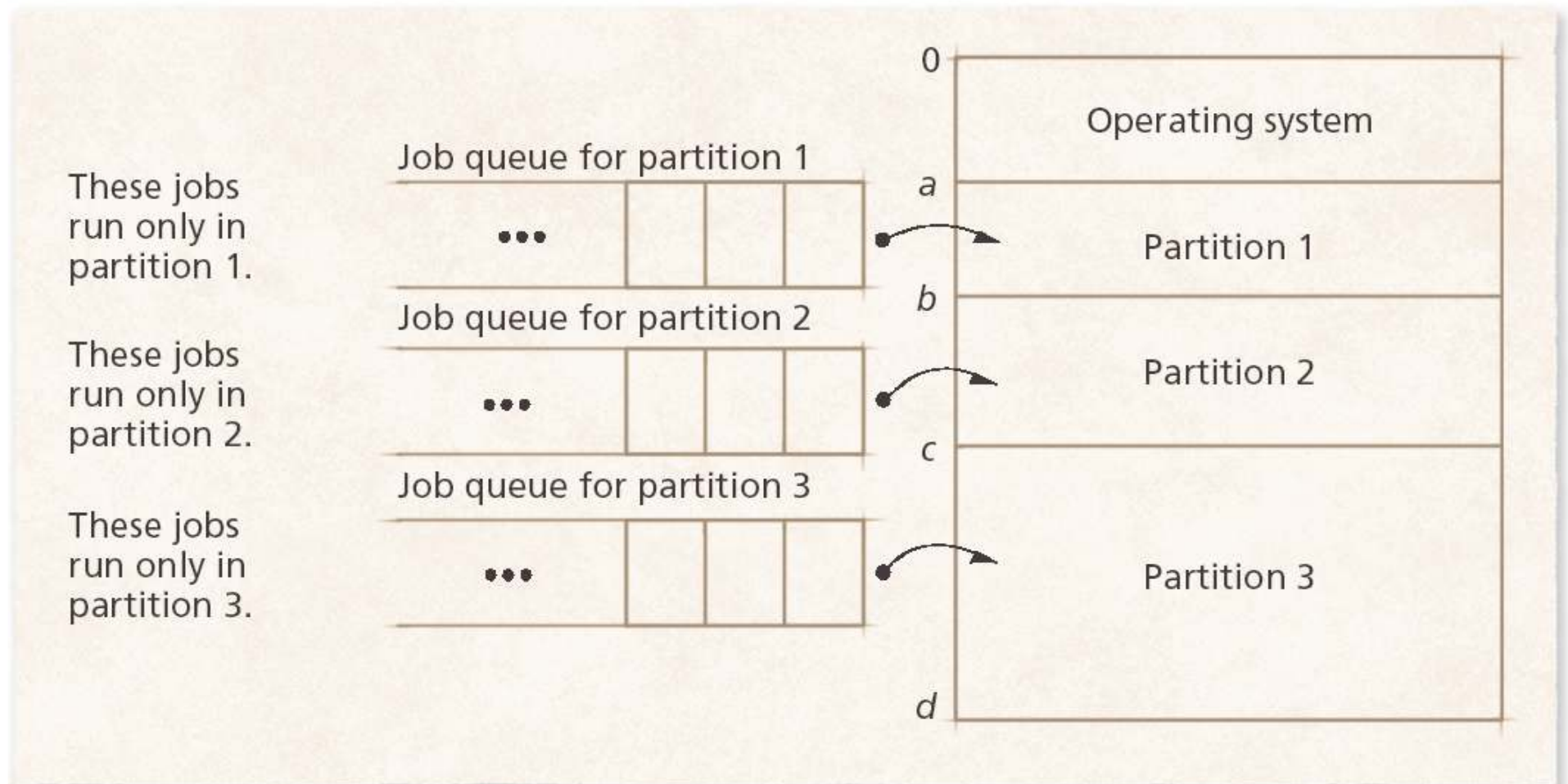


Fixed Partitions

- **Fixed Partitions:** Main memory is partitioned; one partition/job
 - Allows multiprogramming
 - Partition sizes remain static unless and until computer system is shut down, reconfigured, and restarted
 - Requires protection of the job's memory space
 - Requires matching job size with partition size

Fixed-Partition Multiprogramming

Fixed-partition multiprogramming with absolute translation and loading.

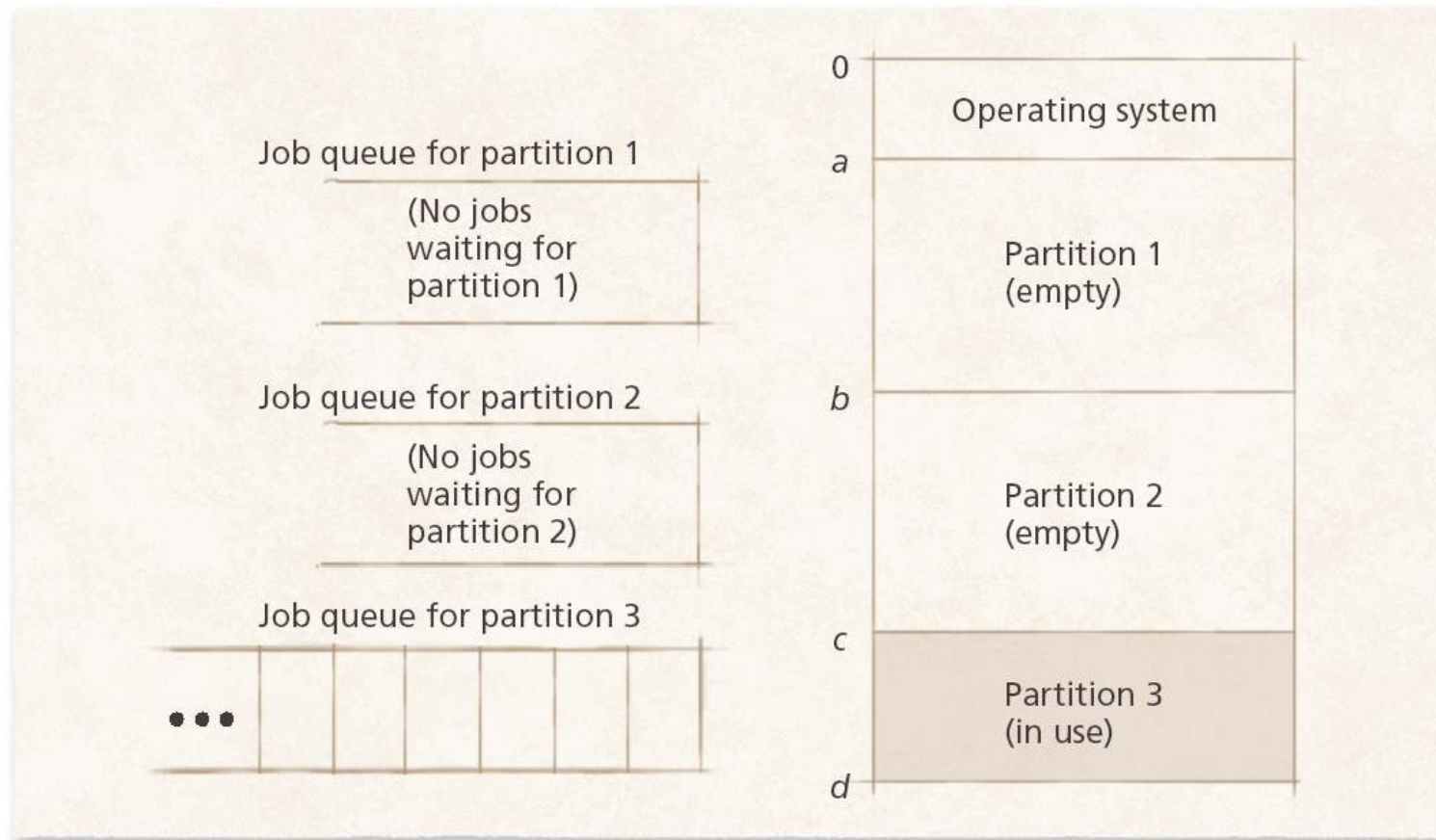


Fixed-Partition Multiprogramming

- Drawbacks to fixed partitions
 - Early implementations used absolute addresses
 - ▶ If the requested partition was full, code could not load
 - ▶ Later resolved by relocating compilers

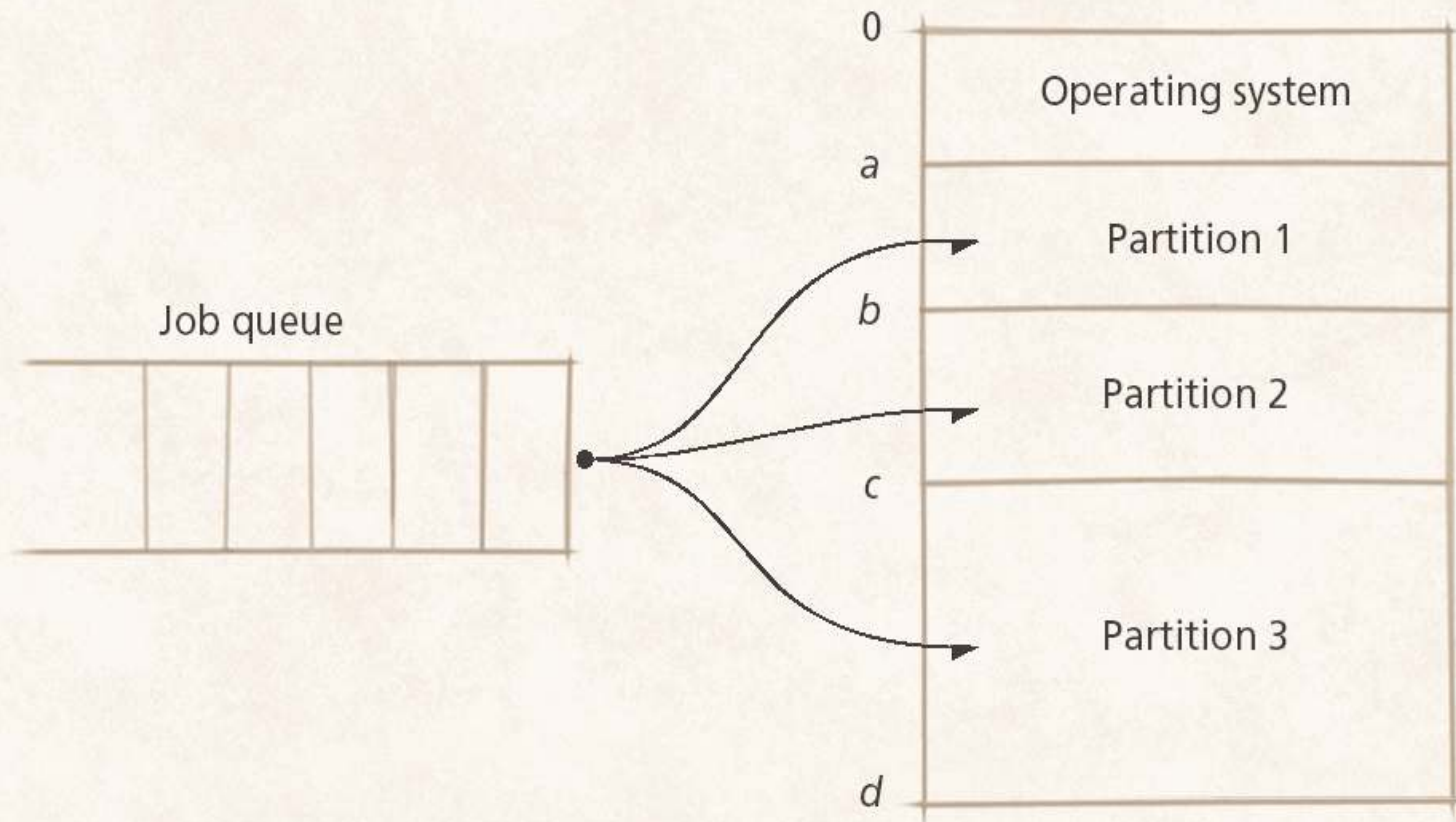
Fixed-Partition Multiprogramming

Memory waste under fixed-partition multiprogramming with absolute translation and loading.



Fixed-Partition Multiprogramming

Fixed-partition multiprogramming with relocatable translation and loading.



A job may be placed in any available partition in which it fits.

Fixed Partitions (continued)

To allocate memory spaces to jobs, the operating system's Memory Manager must keep a table as shown below:

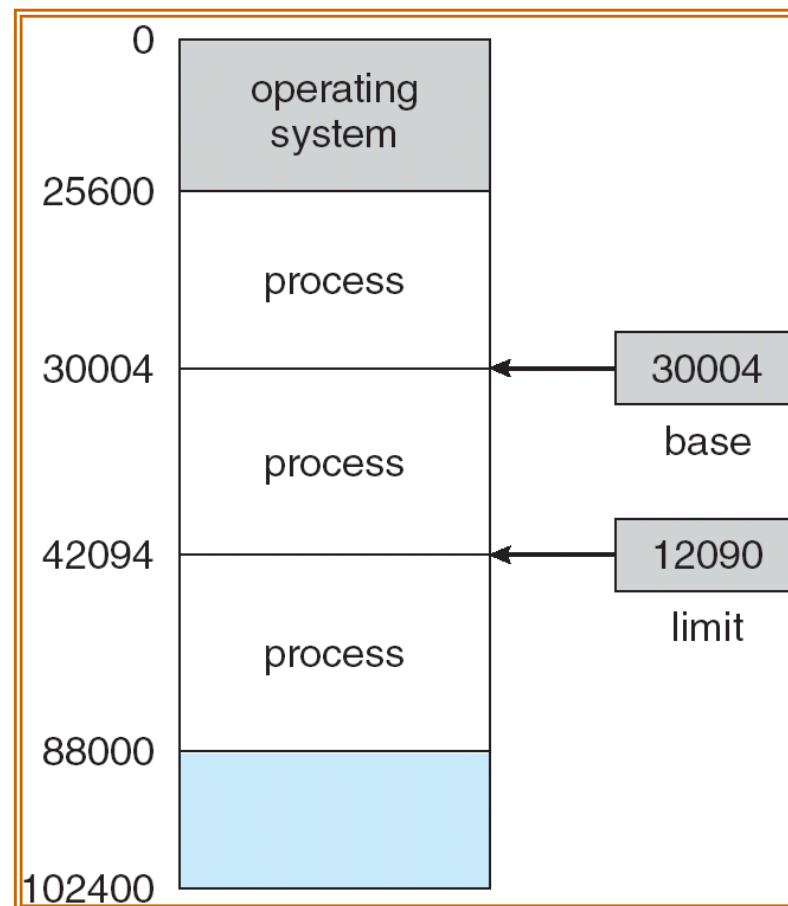
Partition Size	Memory Address	Access	Partition Status
100K	200K	Job 1	Busy
25K	300K	Job 4	Busy
25K	325K		Free
50K	350K	Job 2	Busy

Fixed-Partition Multiprogramming

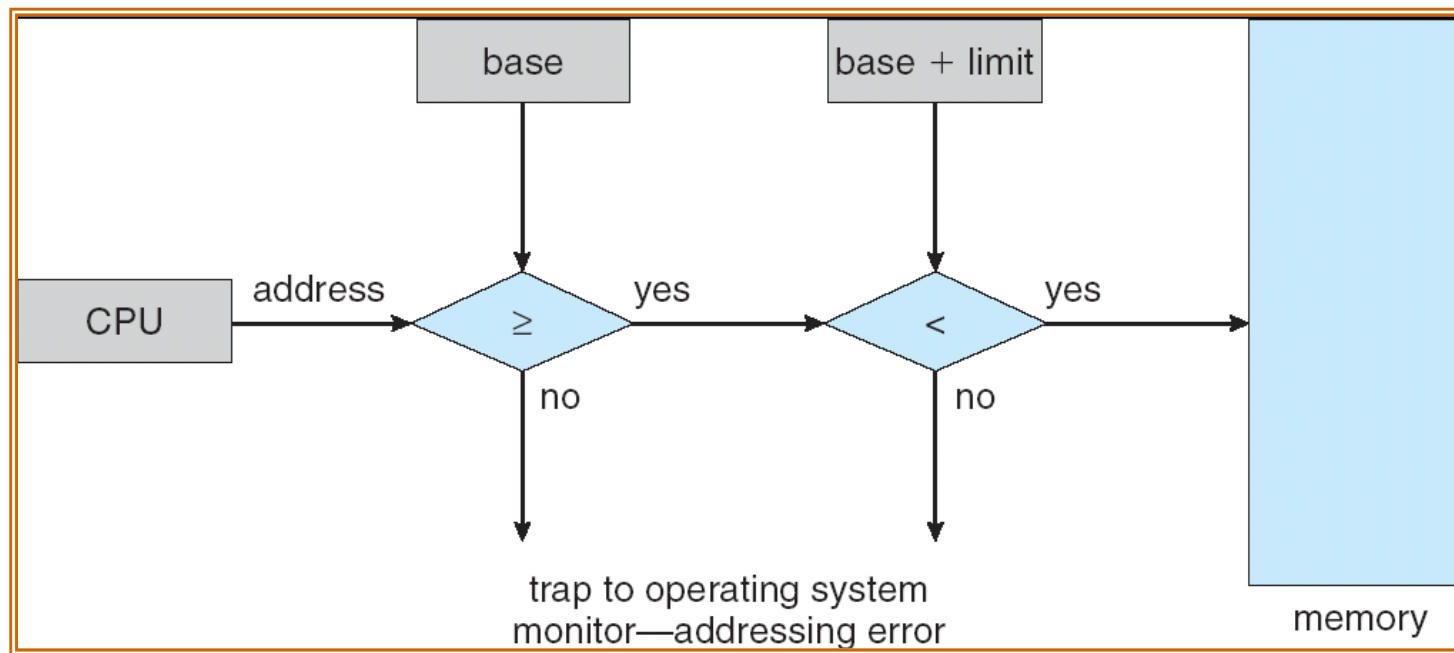
- Protection
 - Can be implemented by boundary registers, called base and limit (also called low and high)

Base and Limit Registers

- A pair of **base** and **limit** registers define the logical address space

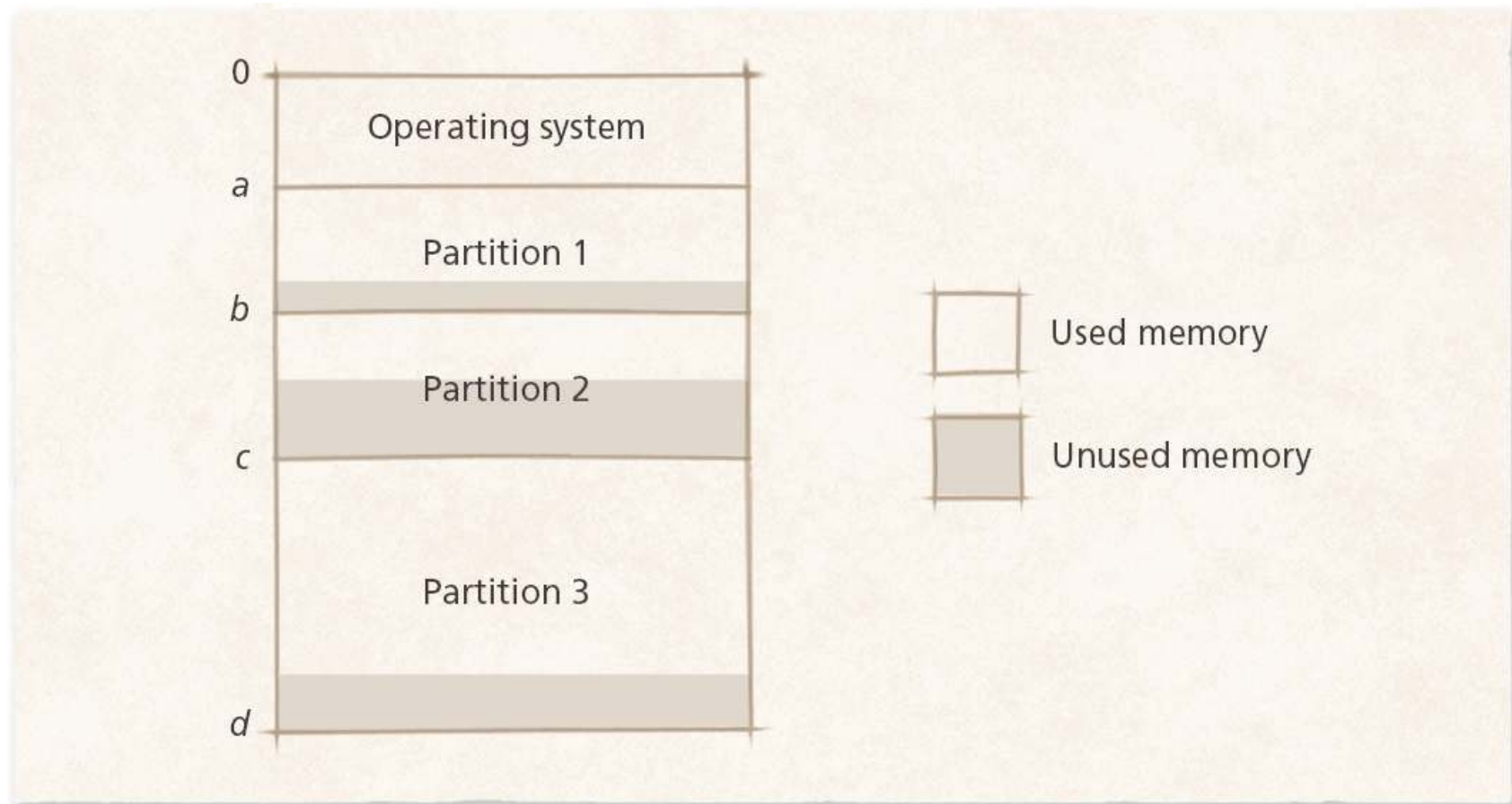


HW address protection with base and limit registers

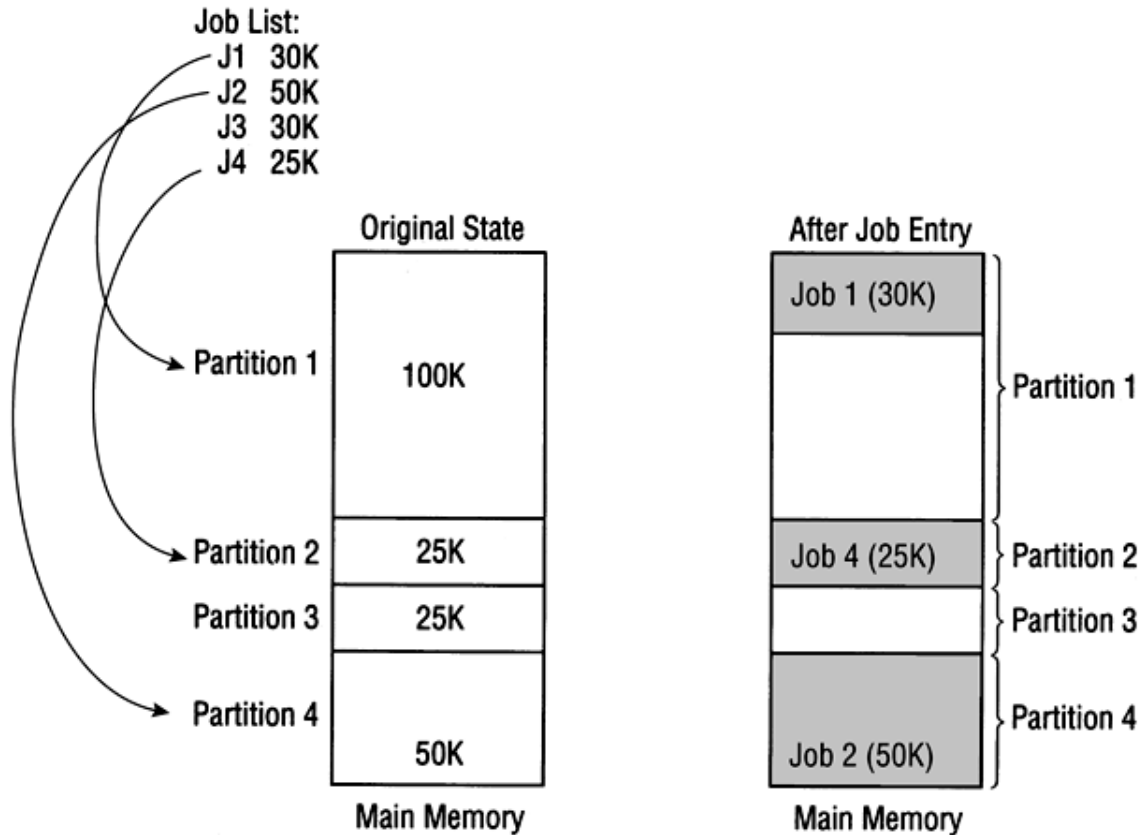


Fixed-Partition Multiprogramming

Internal fragmentation in a fixed-partition multiprogramming system.



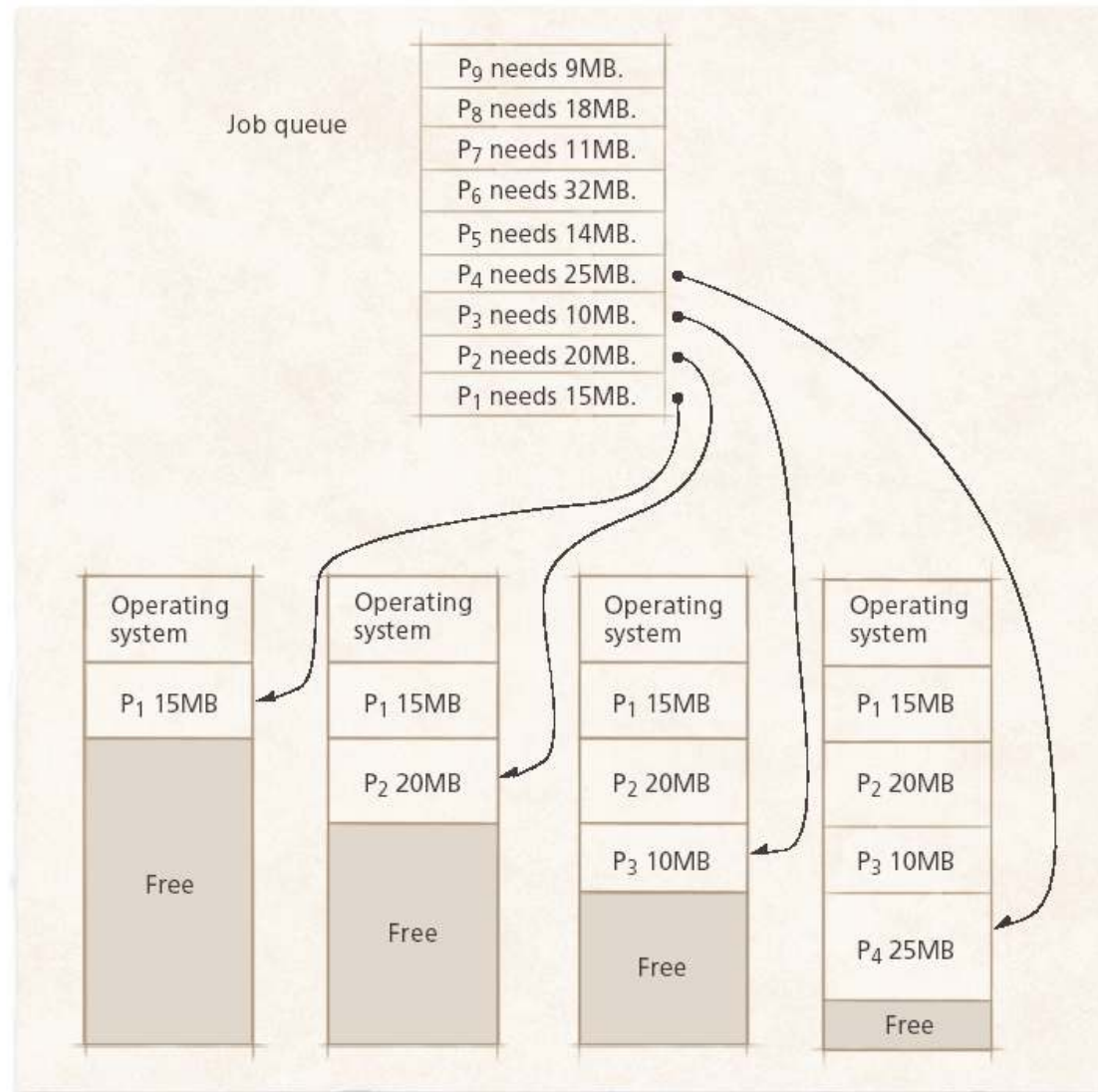
Fixed Partitions (continued)



NOTE: Job 3 must wait even though 70K of free space is available in Partition 1 where Job 1 occupies only 30K of the 100K available

Variable-Partition Multiprogramming

Initial partition assignments in variable-partition programming.

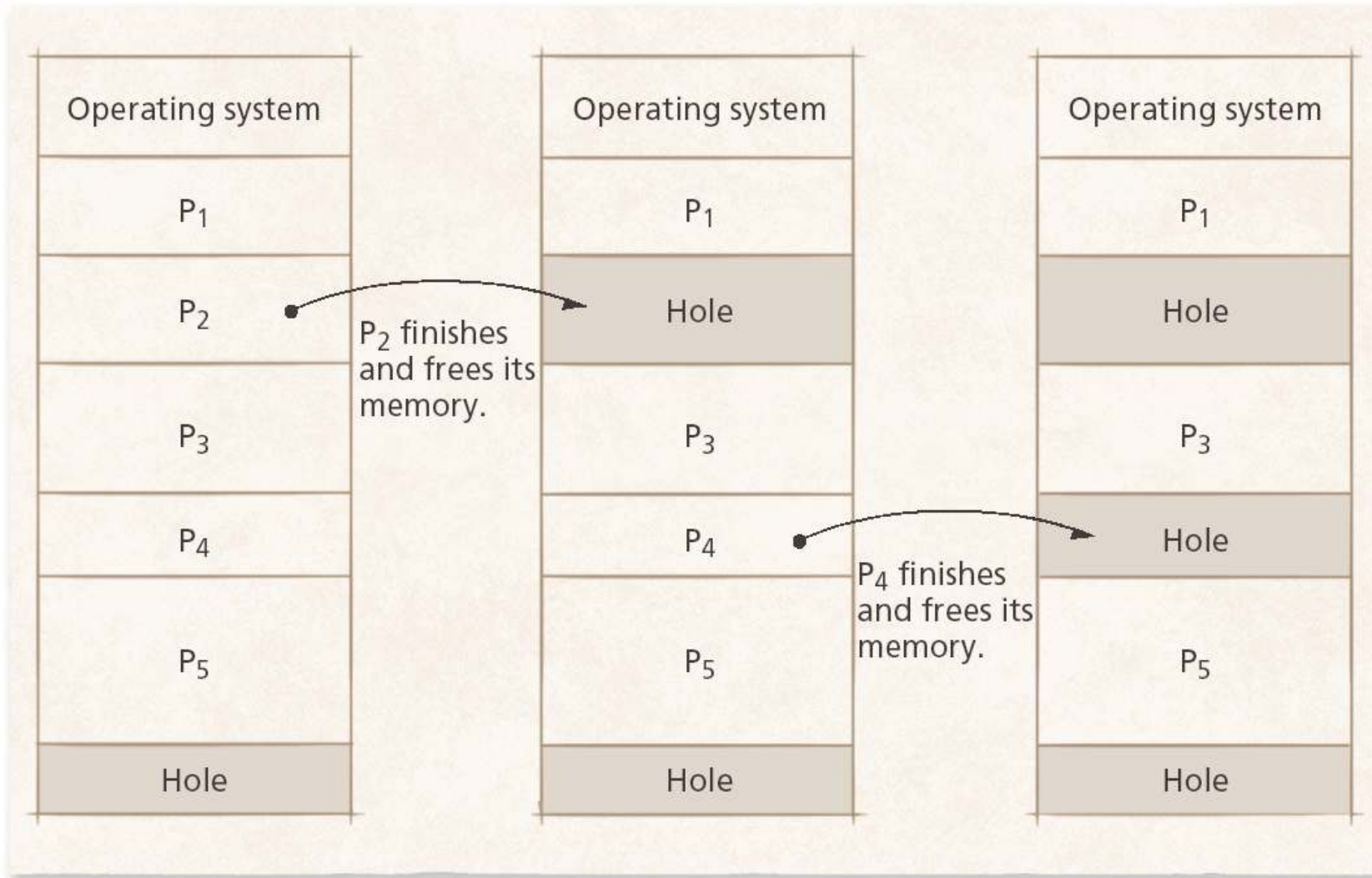


Variable-Partition Characteristics

- Jobs placed where they fit
 - No space wasted initially
 - Internal fragmentation impossible
 - ▶ Partitions are exactly the size they need to be
- External fragmentation can occur when processes removed
 - ▶ Leave holes too small for new processes
 - ▶ Eventually no holes large enough for new processes

Variable-Partition Characteristics

Memory “holes” in variable-partition multiprogramming.

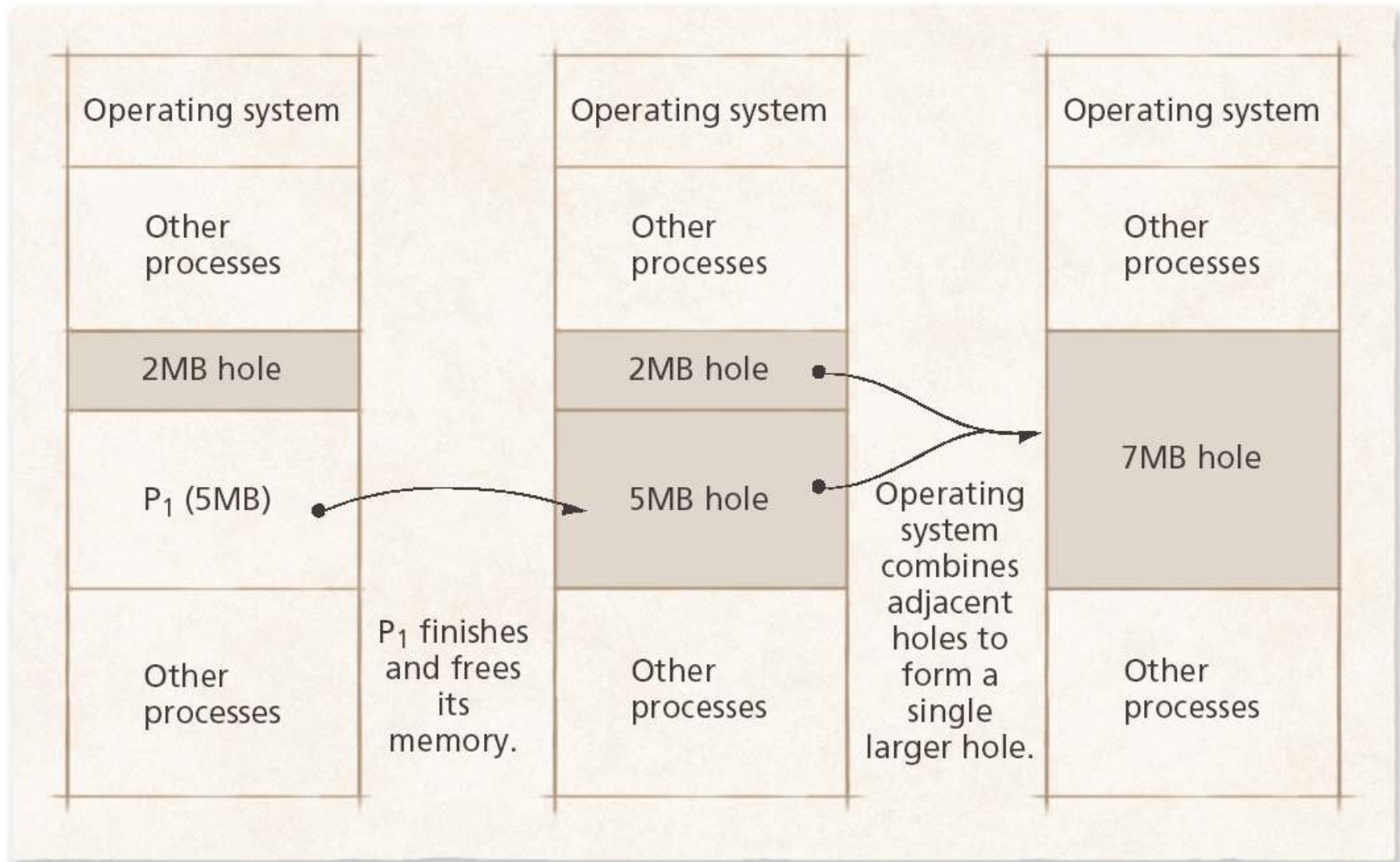


Variable-Partition Characteristics

- Several ways to combat external fragmentation
 - Coalescing
 - ▶ Combine adjacent free blocks into one large block
 - ▶ Often not enough to reclaim significant amount of memory
 - Compaction
 - ▶ Sometimes called garbage collection (not to be confused with GC in object-oriented languages)
 - ▶ Rearranges memory into a single contiguous block free space and a single contiguous block of occupied space
 - ▶ Makes all free space available
 - ▶ Significant overhead
 - ▶ Compaction is possible *only* if relocation is dynamic, and is done at execution time

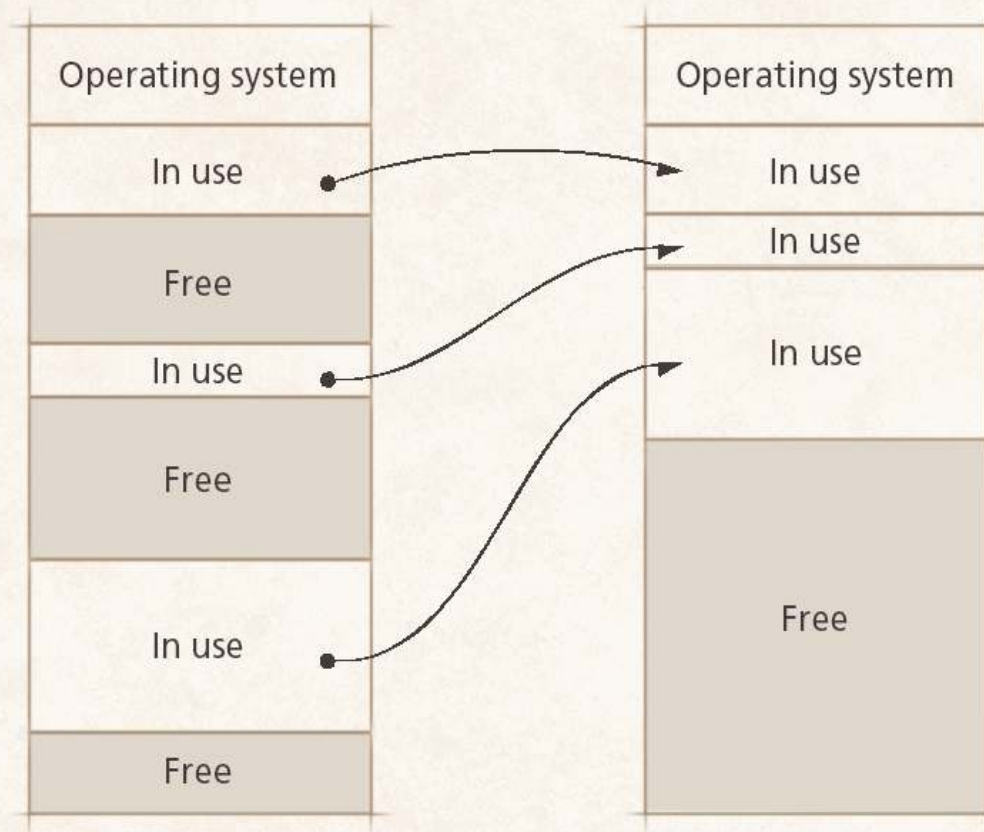
Variable-Partition Characteristics

Coalescing memory “holes” in variable-partition multiprogramming.



Variable-Partition Characteristics

Memory compaction in variable-partition multiprogramming.



Operating system places all "in use" blocks together leaving free memory as a single large hole.

Memory Placement Strategies

- Where to put incoming processes
 - First-fit strategy
 - ▶ Process placed in first hole of sufficient size found
 - ▶ Simple, low execution-time overhead
 - Best-fit strategy
 - ▶ Process placed in hole that leaves least unused space around it
 - ▶ More execution-time overhead
 - Worst-fit strategy
 - ▶ Process placed in hole that leaves most unused space around it
 - ▶ Leaves another large hole, making it more likely that another process can fit in the hole

Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 1 of 3).

(a) First-fit strategy

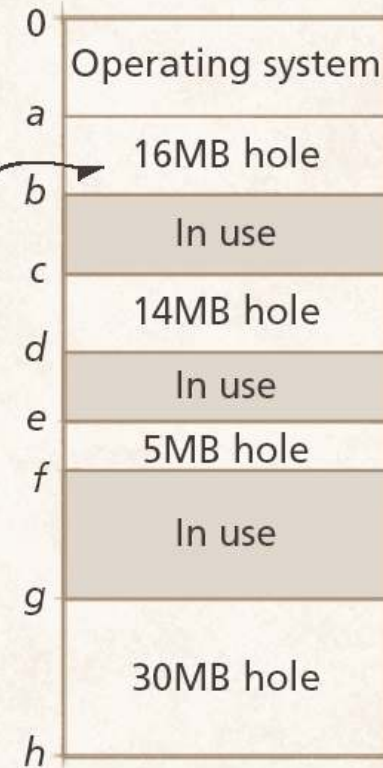
Place job in first memory hole on free memory list in which it will fit.

Free Memory List (Kept in random order.)

Start address *Length*

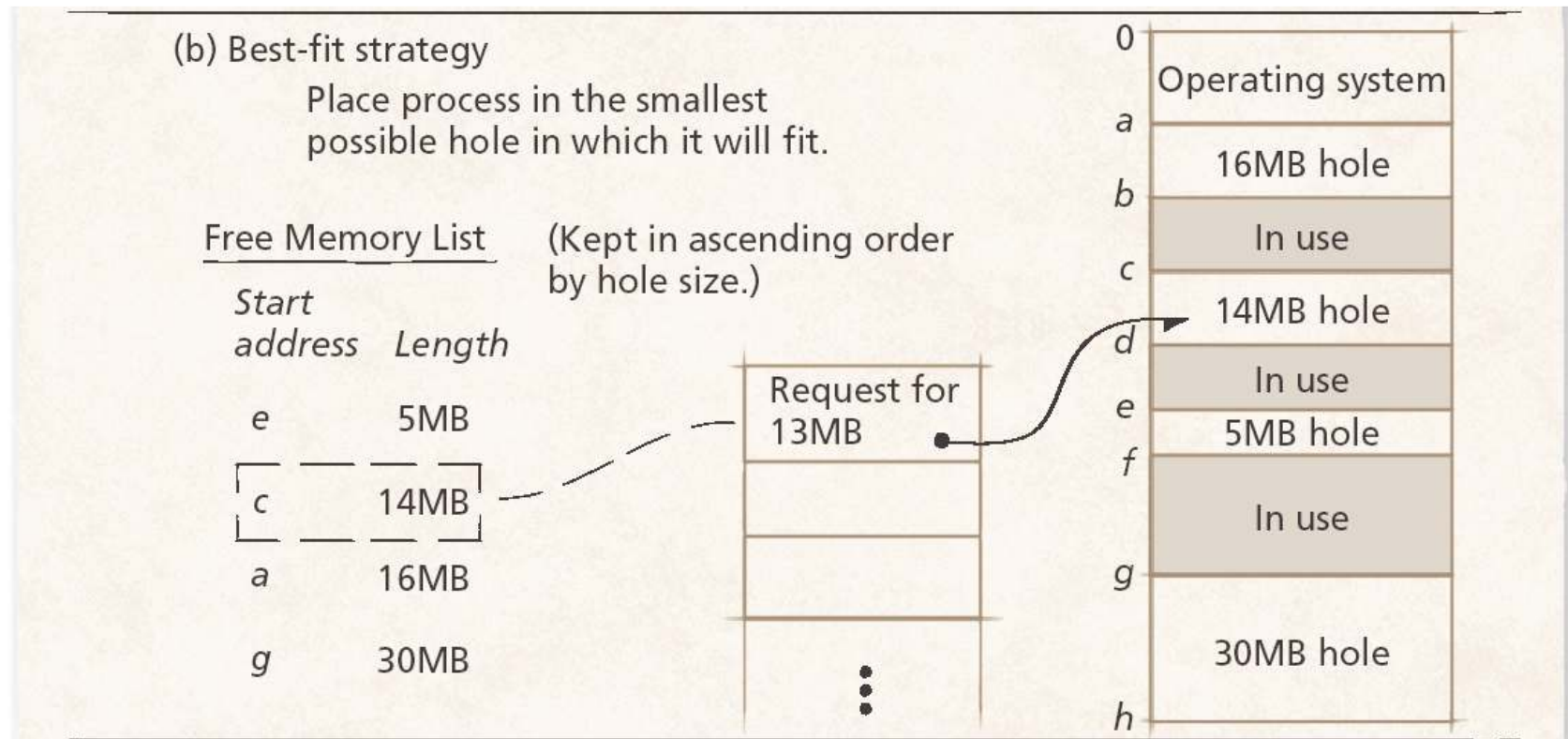
a	16MB
e	5MB
c	14MB
g	30MB

Request for
13MB



Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 2 of 3).



Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 3 of 3).

(c) Worst-fit strategy

Place process in the largest possible hole in which it will fit.

Free Memory List

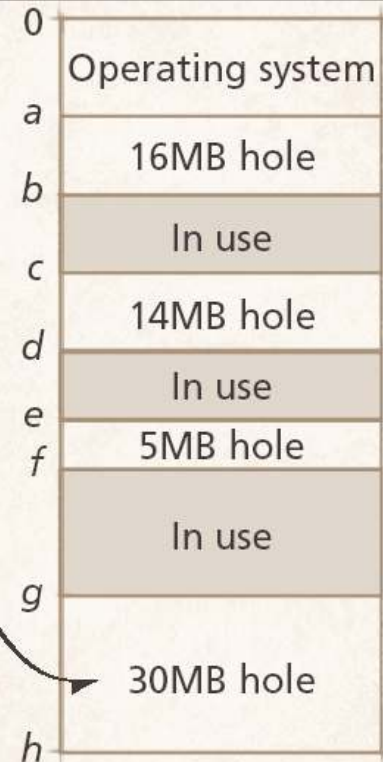
(Kept in descending order by hole size.)

Start address *Length*

<i>g</i>	30MB
<i>a</i>	16MB
<i>c</i>	14MB
<i>e</i>	5MB

Request for 13MB

⋮



Best-Fit Versus First-Fit Allocation (continued)

- **First-fit memory allocation:**
 - **Advantage:** Faster in making allocation
 - **Disadvantage:** Leads to memory waste

- **Best-fit memory allocation**
 - **Advantage:** Makes the best use of memory space
 - **Disadvantage:** Slower in making allocation

Variable Partitions

- **Disadvantages:**

- Fully utilizes memory only when the first jobs are loaded
- Subsequent allocation leads to memory waste or **external fragmentation**

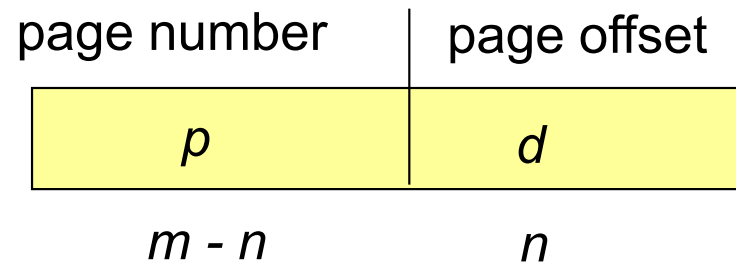
Noncontiguous Scheme

Paging

- ❑ Paging is noncontiguous memory allocation scheme
- ❑ Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8,192 bytes)
- ❑ Divide logical memory into blocks of same size called **pages**
- ❑ Keep track of all free frames
- ❑ To run a program of size n pages, need to find n free frames and load program
- ❑ Set up a page table to translate logical to physical addresses
- ❑ Internal fragmentation

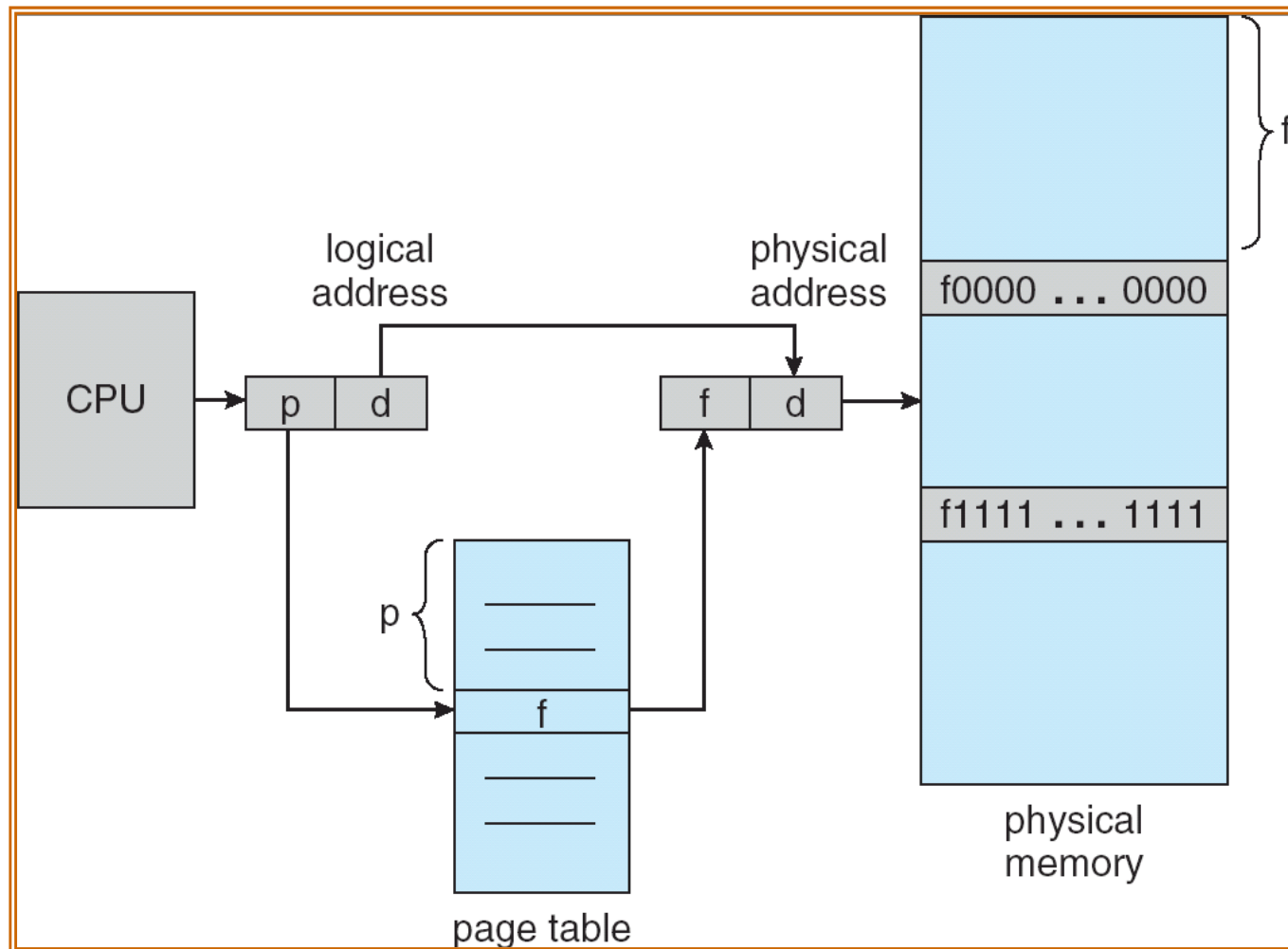
Address Translation Scheme

- Address (m bits) generated by CPU is divided into:
 - **Page number (p)** – used as an index into a *page table* which contains base address of each page in physical memory
 - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit

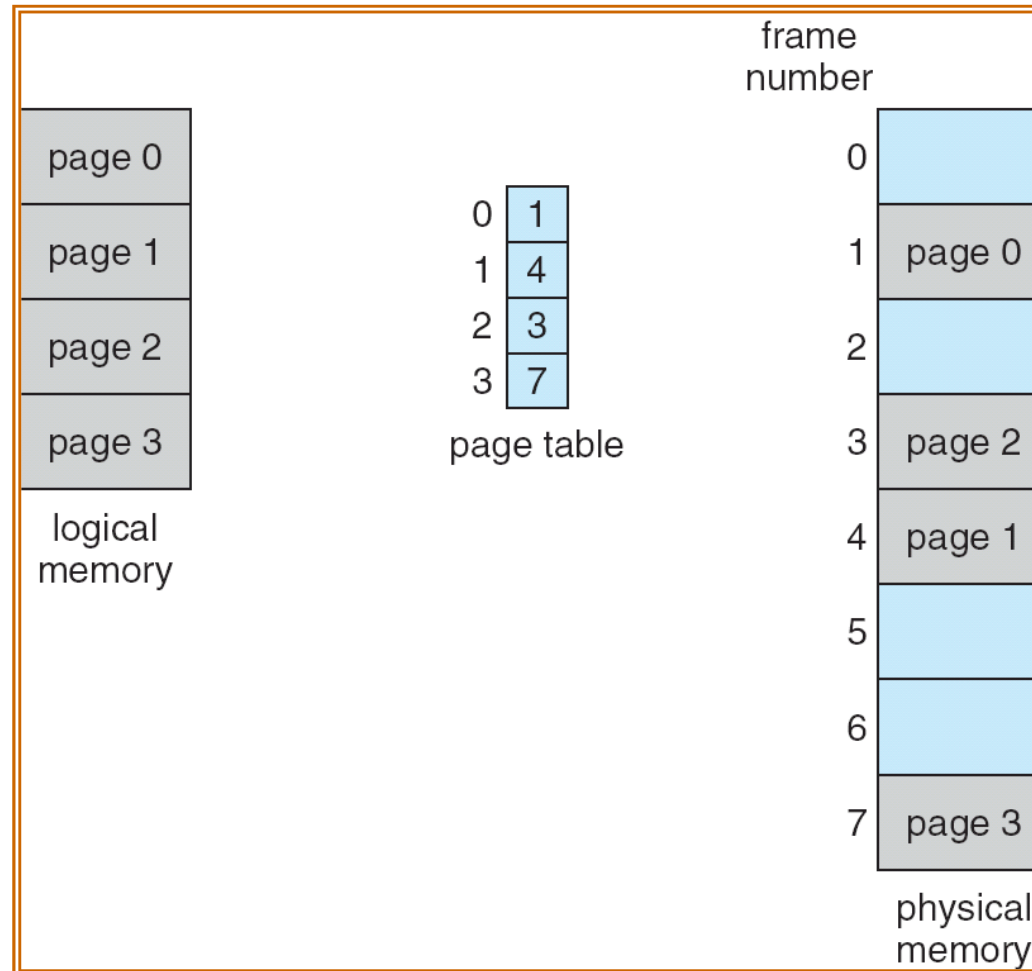


- For given logical address space 2^m and page size 2^n

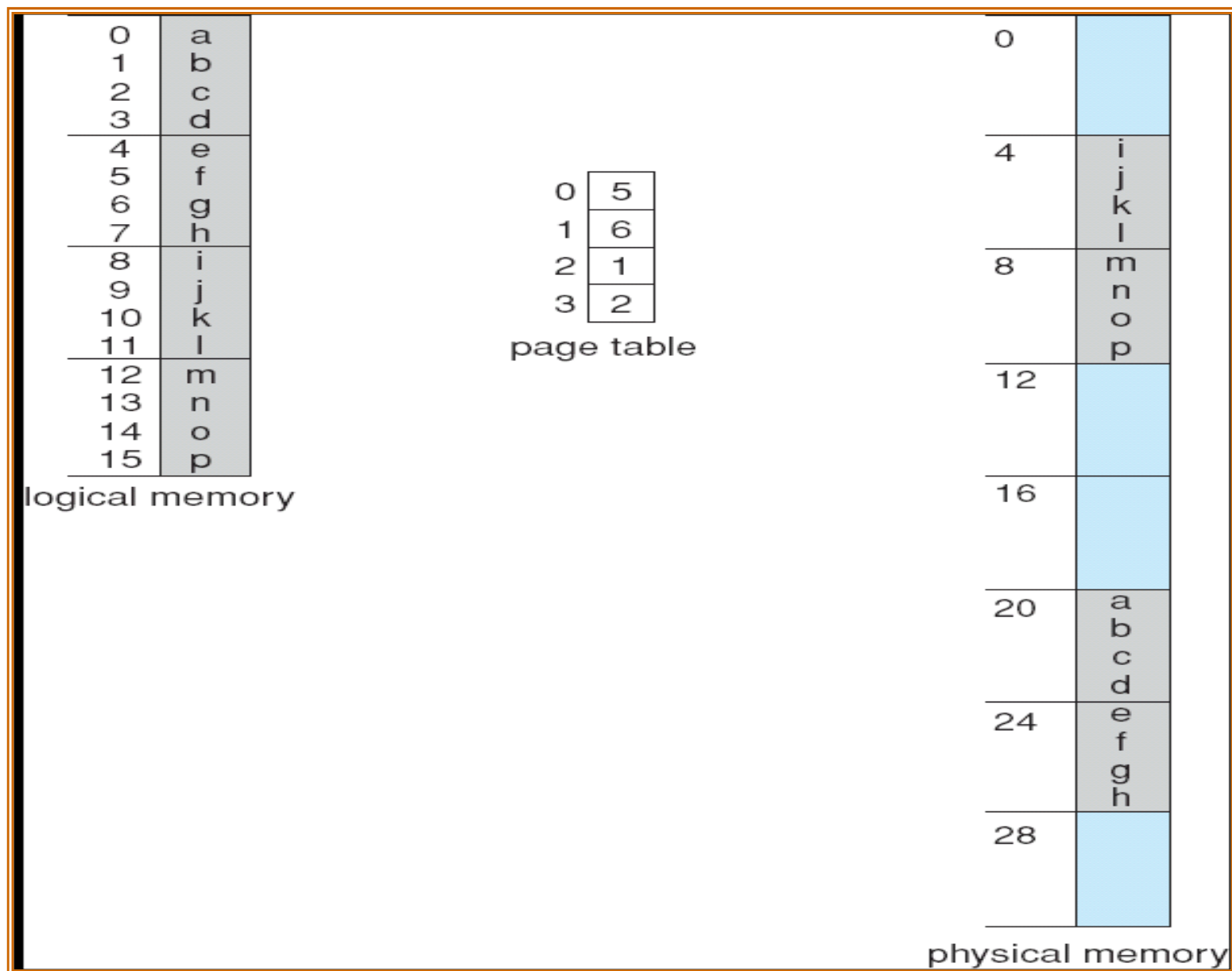
Paging Hardware



Paging Model of Logical and Physical Memory

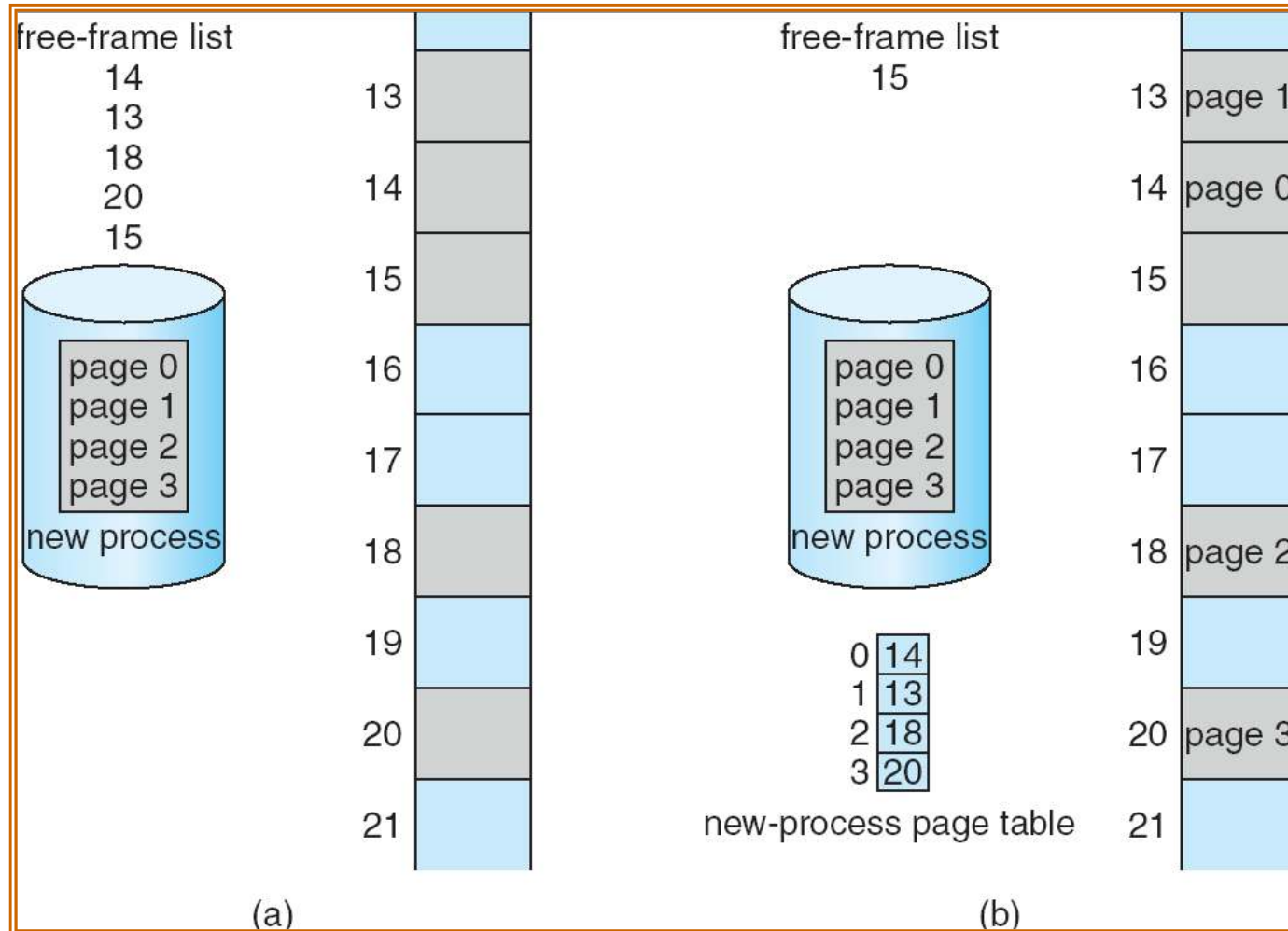


Paging Example



32-byte memory and 4-byte pages

Free Frames



Before allocation

After allocation

Implementation of Page Table

- ❑ Page table is kept in main memory
- ❑ **Page-table base register (PTBR)** points to the page table
- ❑ **Page-table length register (PTLR)** indicates size of the page table
- ❑ In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.
- ❑ The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

Associative Memory

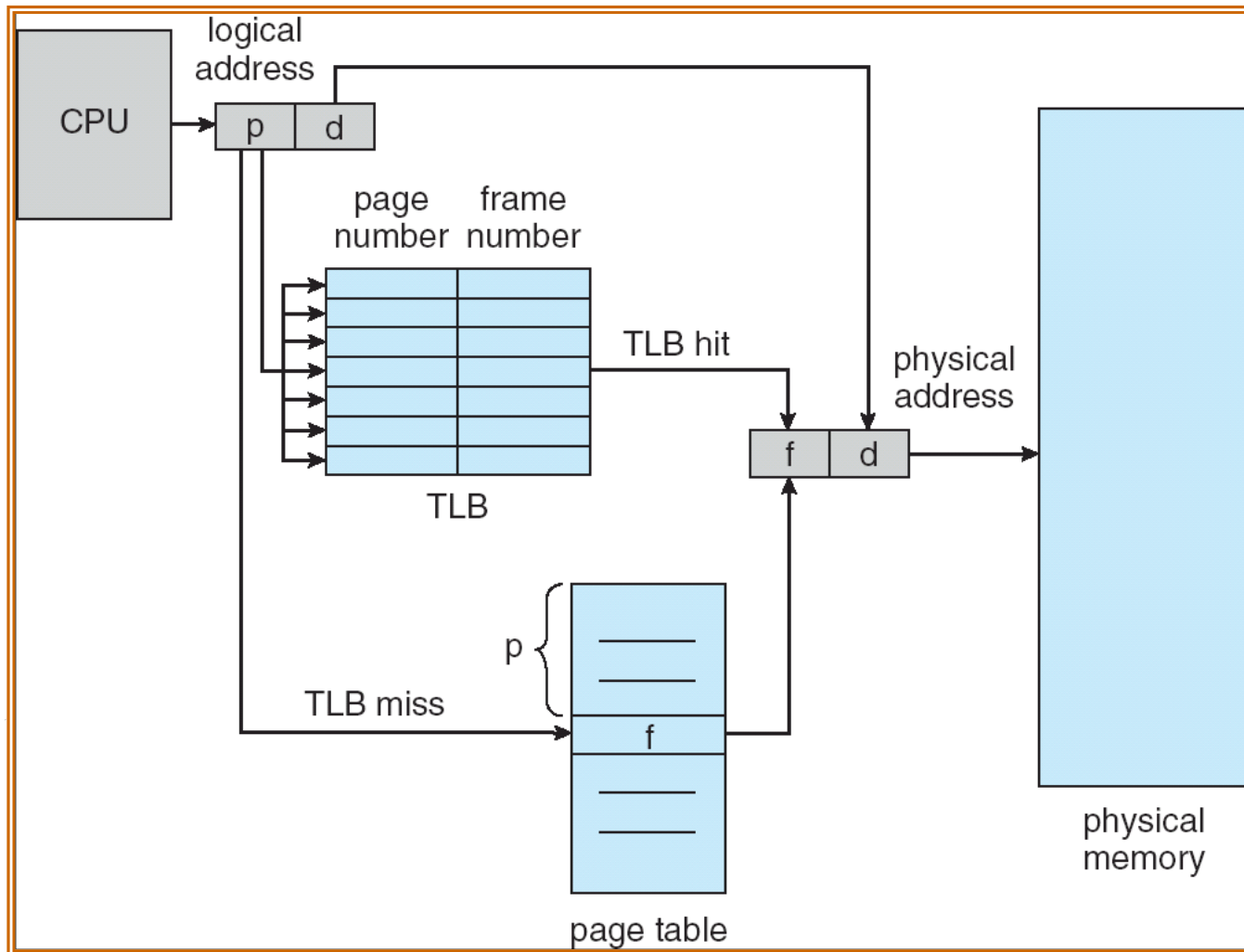
- Associative memory – parallel search

Page #	Frame #

Address translation (p, d)

- If p is in associative register, get frame # out
- Otherwise get frame # from page table in memory

Paging Hardware With TLB



Effective Access Time

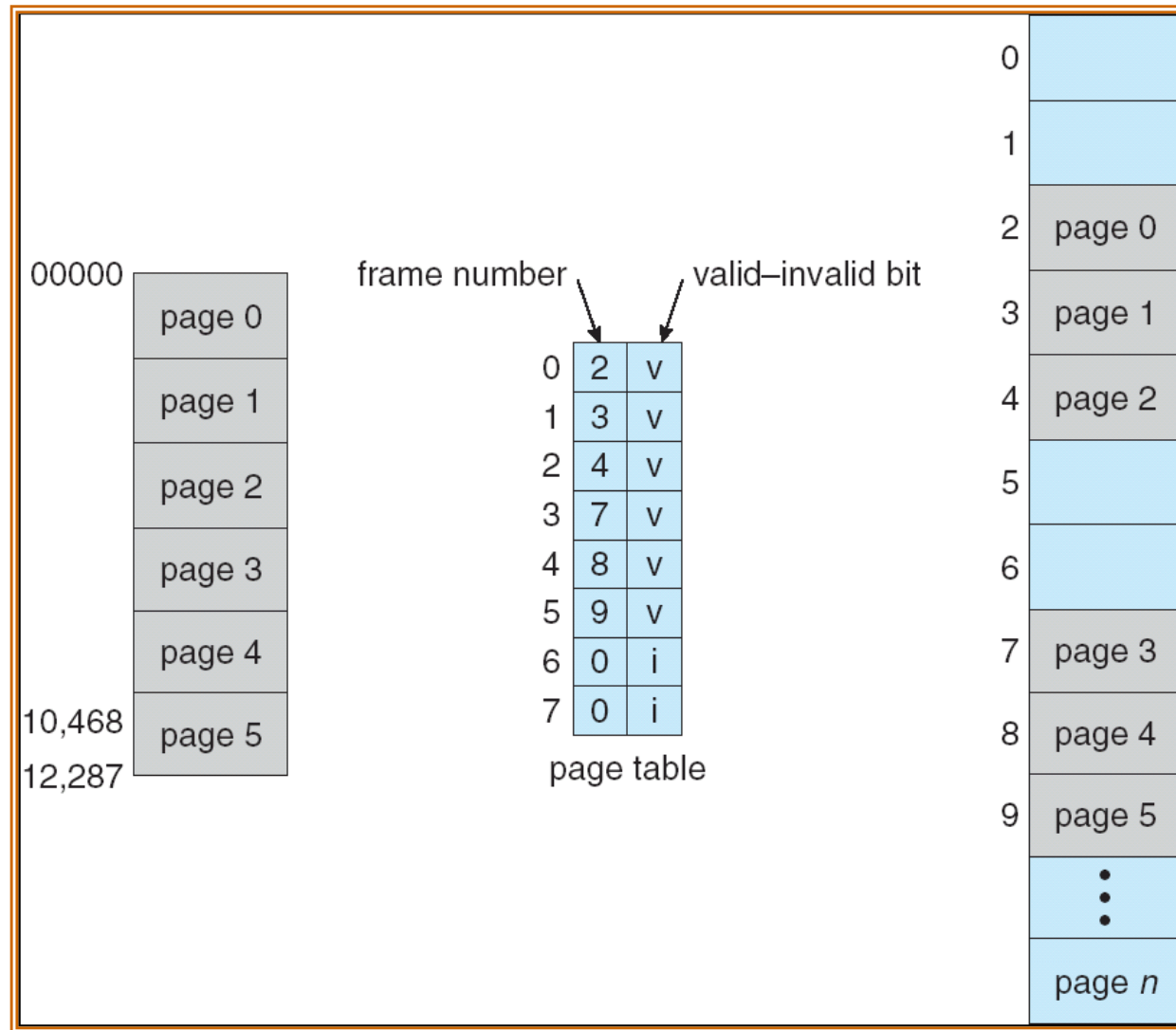
- Associative Lookup = ε time unit
- Assume memory cycle time is 1 microsecond
- Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers
- Hit ratio = α
- **Effective Access Time (EAT)**

$$\begin{aligned} \text{EAT} &= (1 + \varepsilon) \alpha + (2 + \varepsilon)(1 - \alpha) \\ &= 2 + \varepsilon - \alpha \end{aligned}$$

Memory Protection

- Memory protection implemented by associating protection bit with each frame
- **Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space

Valid (v) or Invalid (i) Bit In A Page Table



Shared Pages

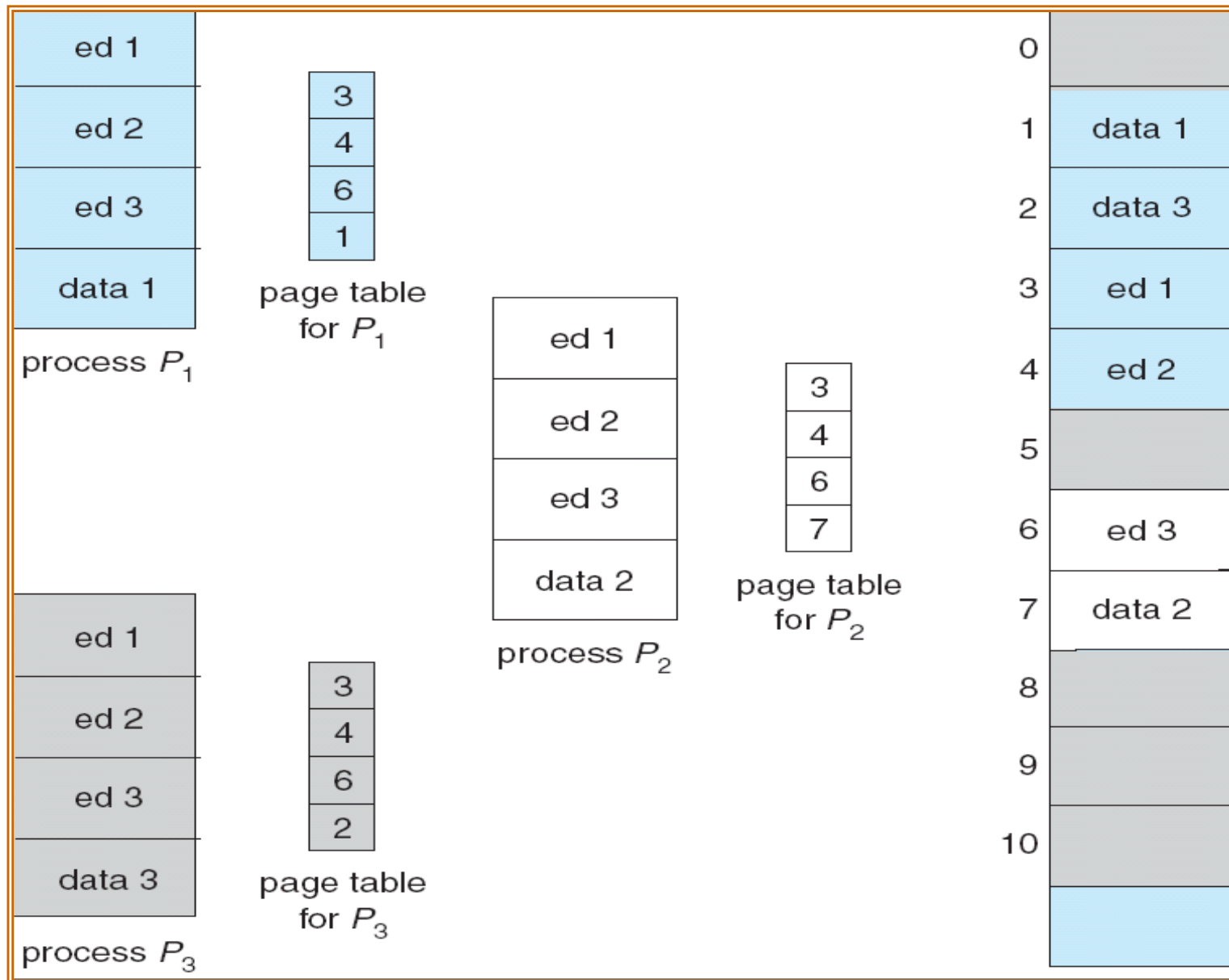
□ **Shared code**

- One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems).
- Shared code must appear in same location in the logical address space of all processes

□ **Private code and data**

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example



Structure of the Page Table

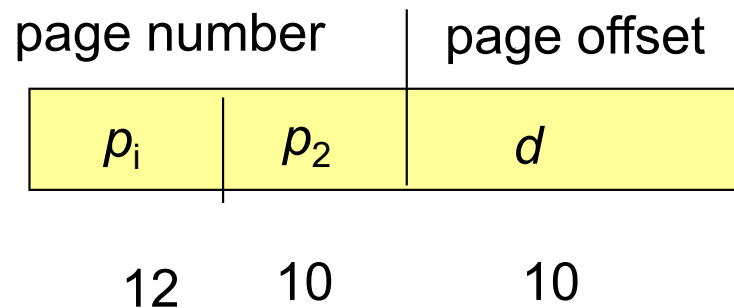
- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table

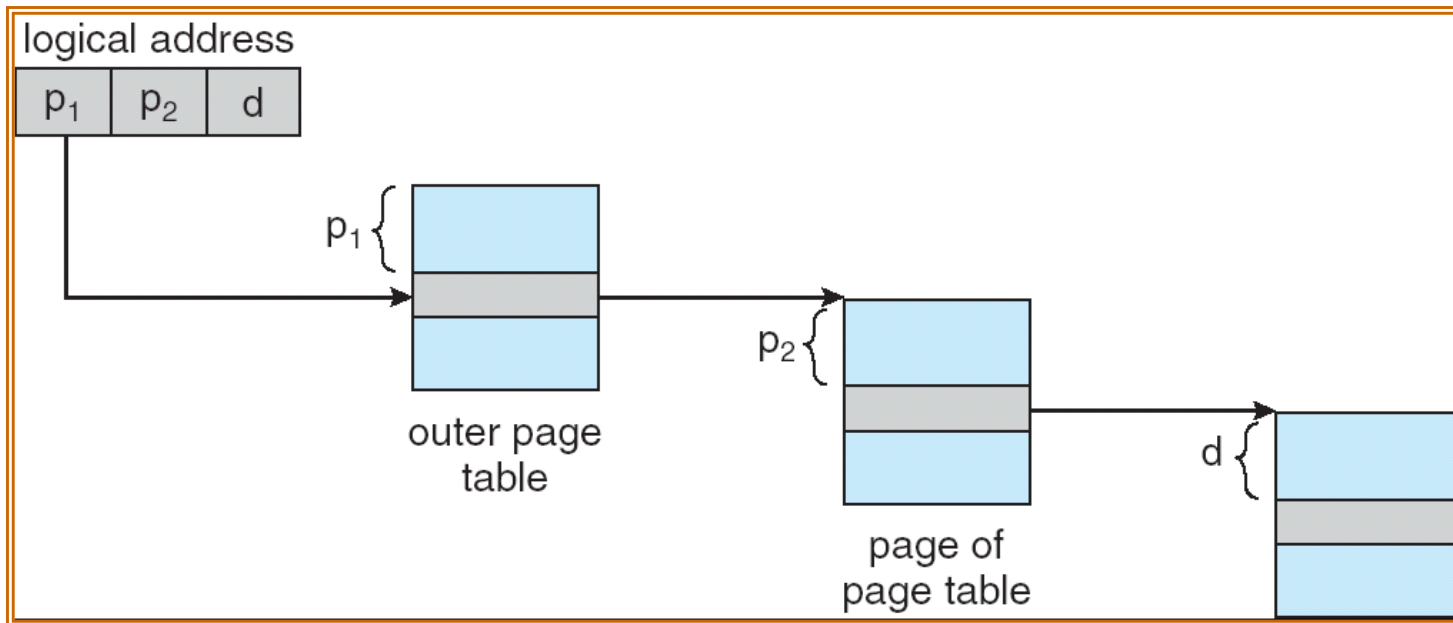
Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
 - a page number consisting of 22 bits
 - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
 - a 12-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:



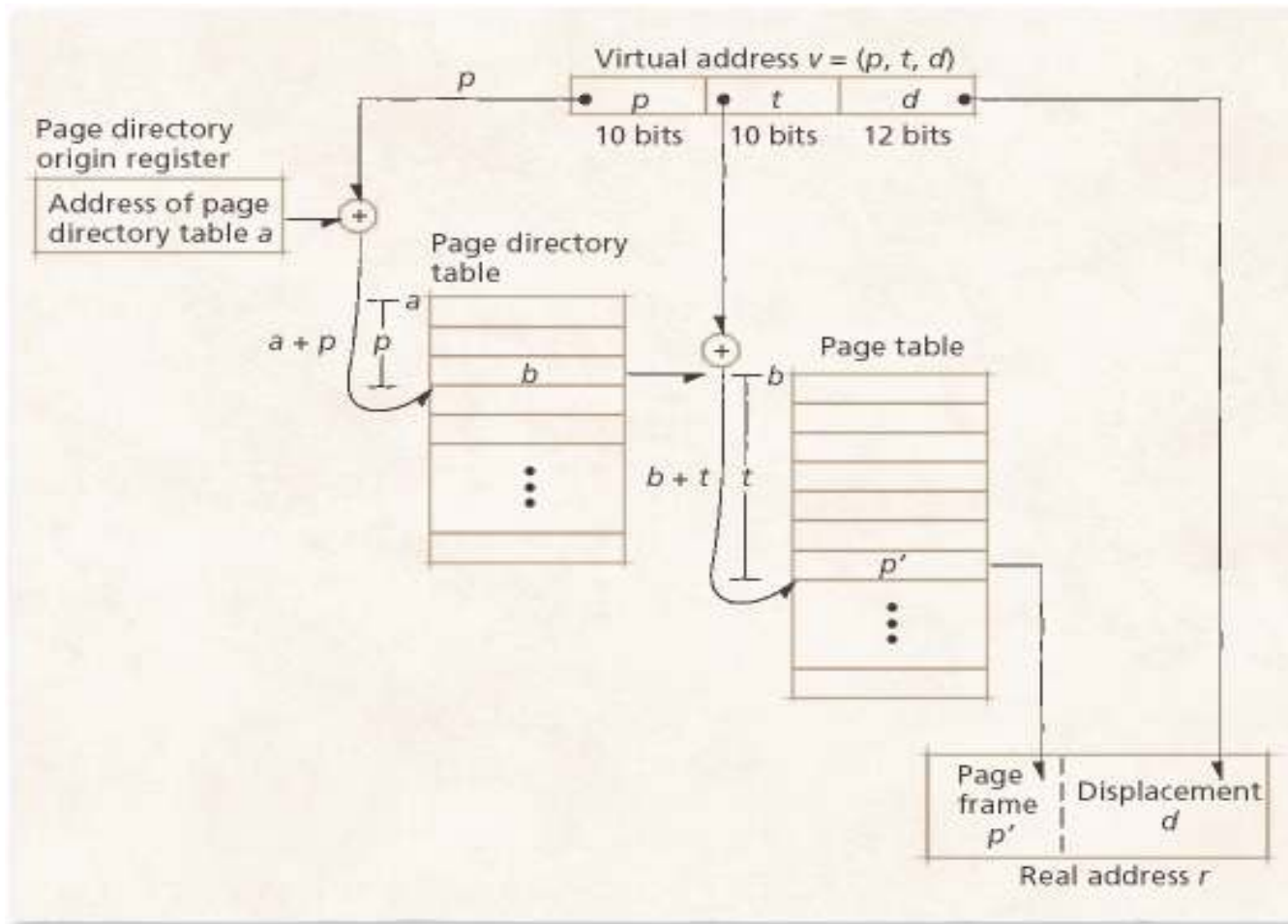
where p_i is an index into the outer page table, and p_2 is the displacement within the page of the outer page table

Address-Translation Scheme



Multilevel Page Tables

Multilevel page address translation.



Three-level Paging Scheme

outer page	inner page	offset
p_1	p_2	d
42	10	12

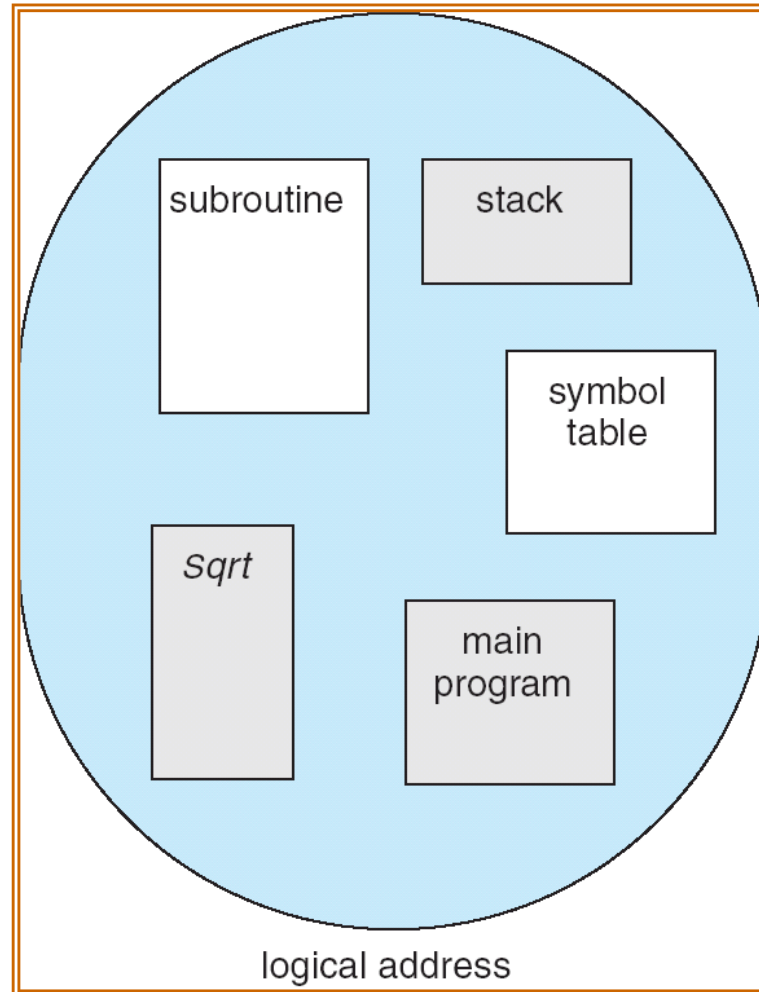
2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

Segmentation

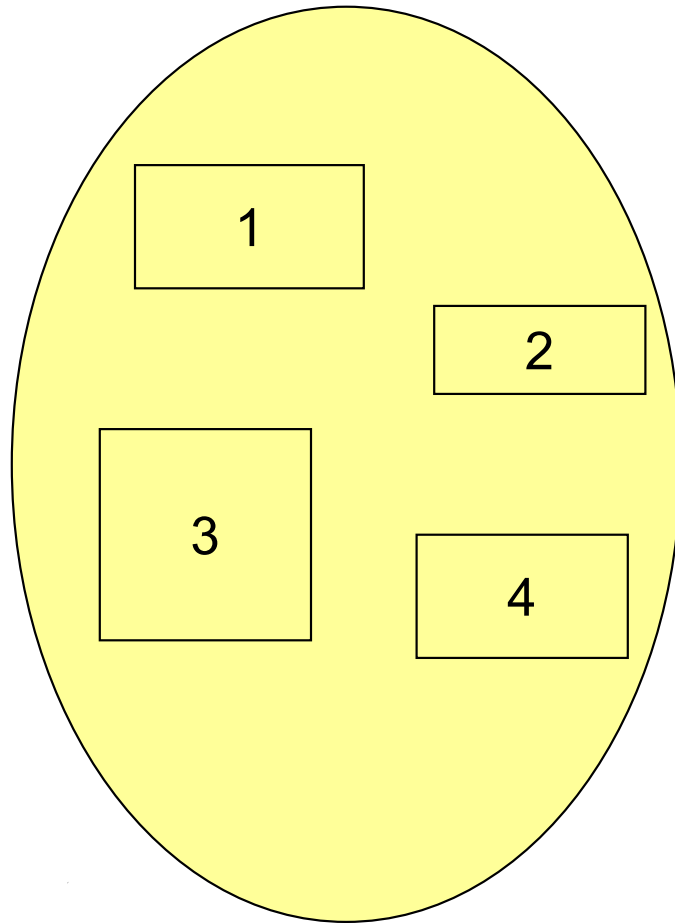
- Memory-management scheme that supports user view of memory
- A program is a collection of segments. A segment is a logical unit such as:

main program,
procedure,
function,
method,
object,
local variables, global variables,
common block,
stack,
symbol table, arrays

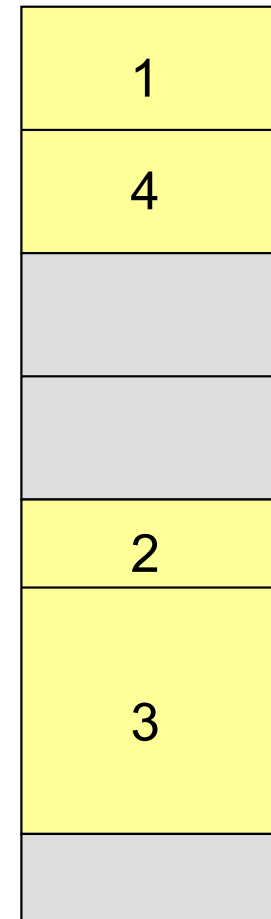
User's View of a Program



Logical View of Segmentation



user space

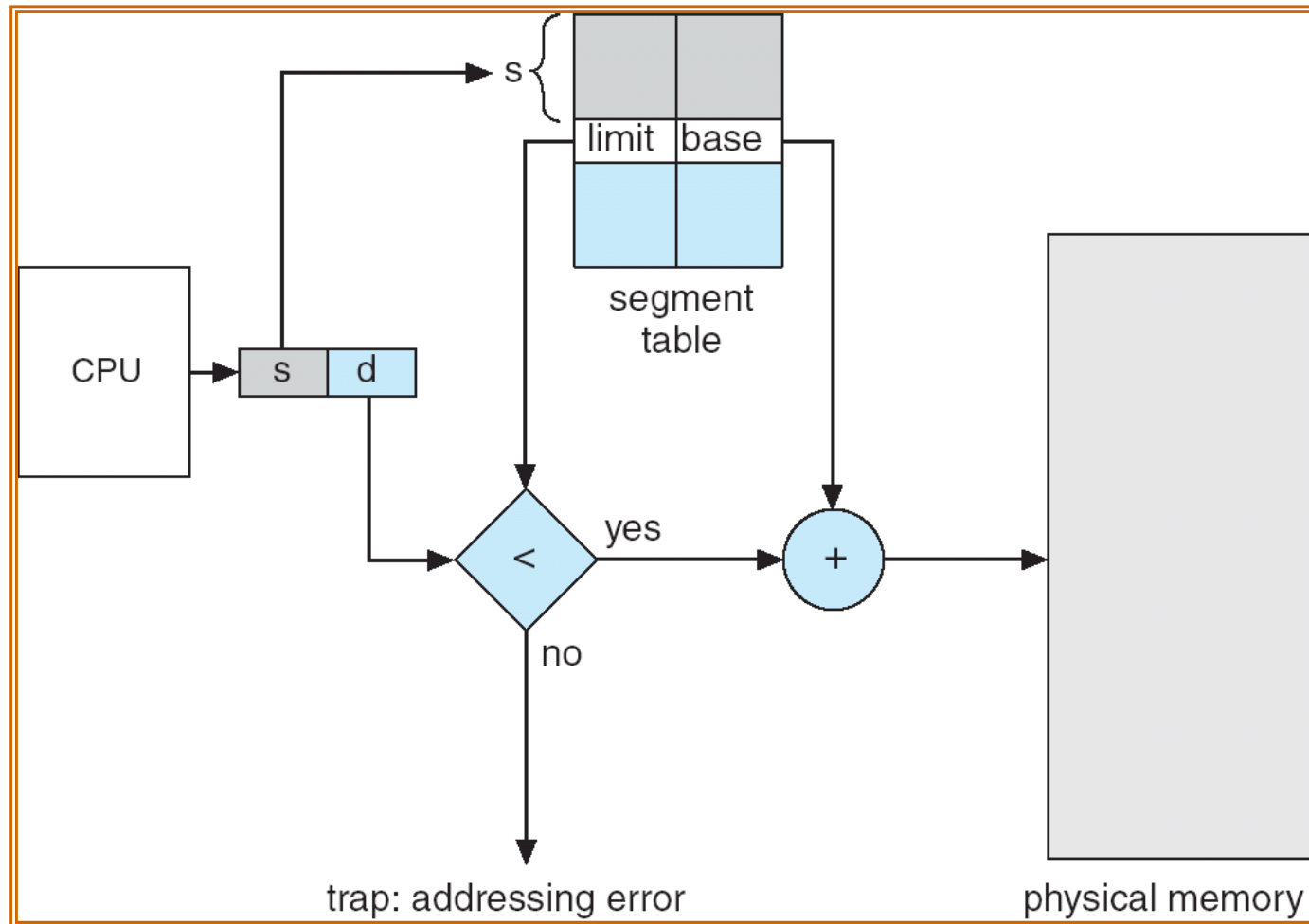


physical memory space

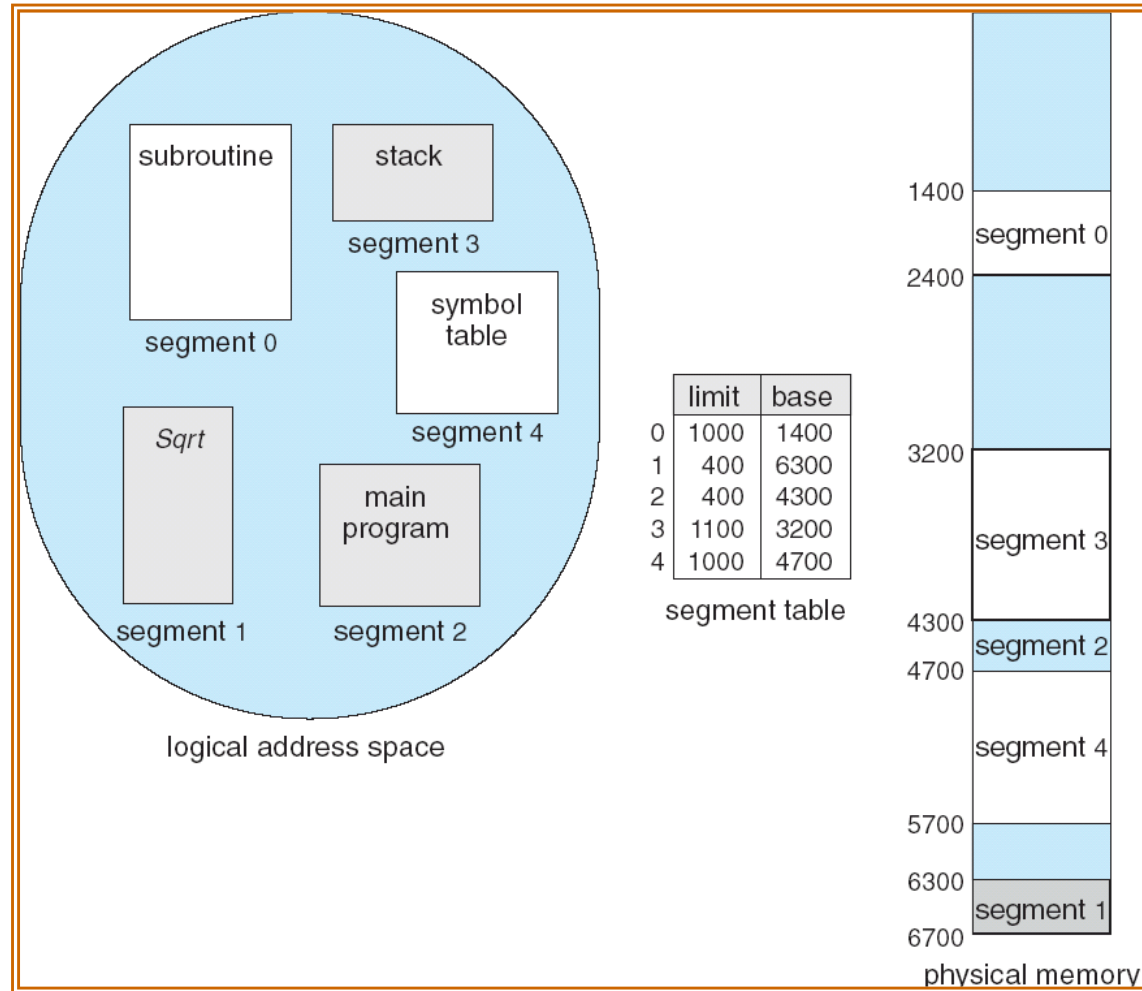
Segmentation Architecture

- Logical address consists of a two tuple:
 <segment-number, offset>,
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - **base** – contains the starting physical address where the segments reside in memory
 - **limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;
 segment number **s** is legal if **s** < **STLR**

Segmentation Hardware



Example of Segmentation



Segmentation/Paging Systems

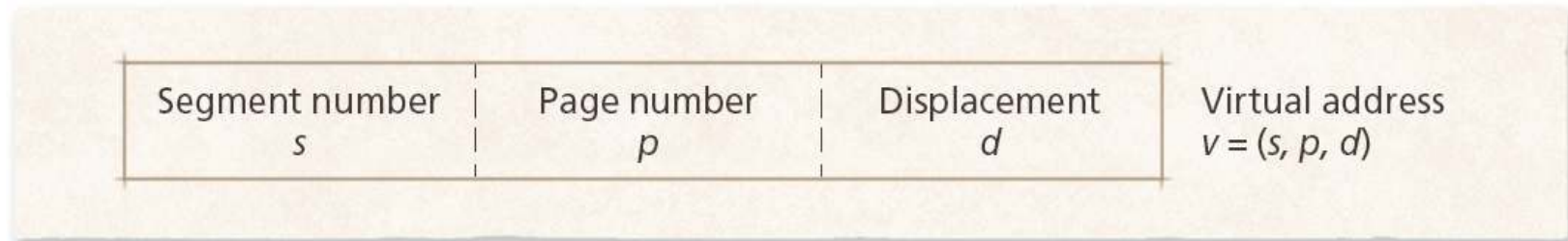
- Process references virtual memory address

$$v = (s, p, d)$$

- DAT adds the process's segment map table base address, b , to referenced segment number, s
- $b + s$ forms the main memory address of the segment map table entry for segment s
- Segment map table entry stores the base address of the page table, s'
- Referenced page number, p , is added to s' to locate the PTE for page p , which stores page frame number p'
- System concatenates p' with displacement, d , to form real address, r

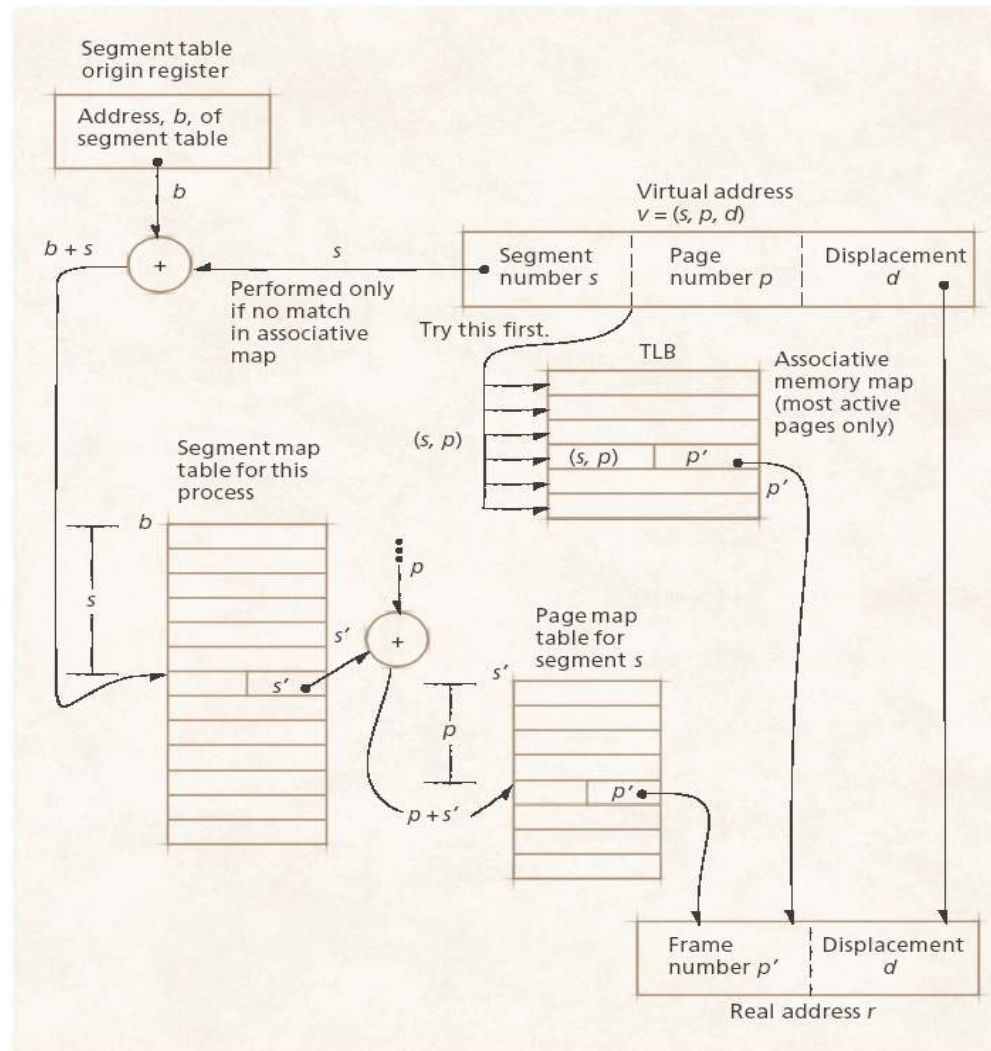
Dynamic Address Translation in a Segmentation/Paging System

Virtual address format in a segmentation/paging system.



Dynamic Address Translation in a Segmentation/Paging System

Virtual address translation with combined associative/direct mapping in a segmentation/paging system.

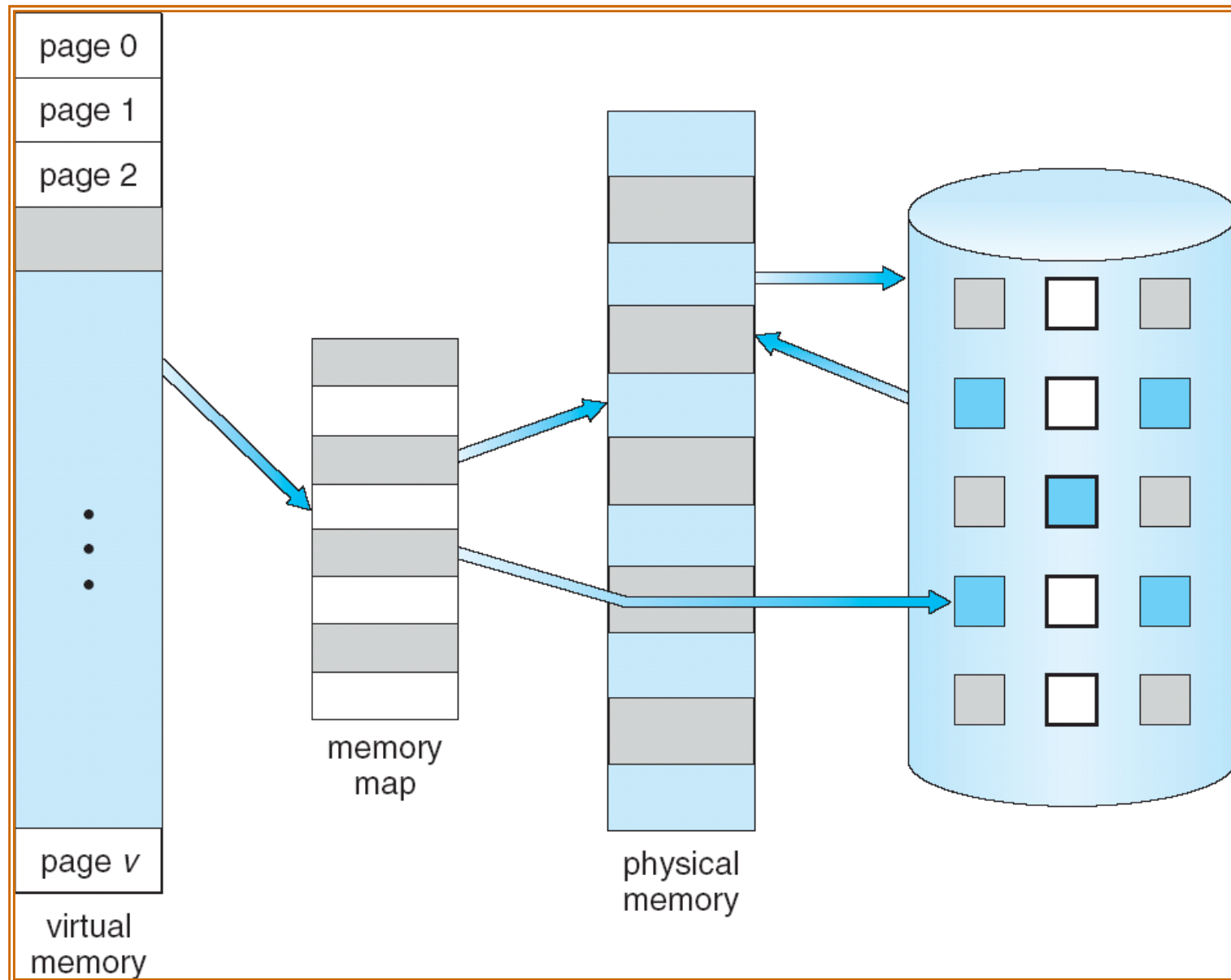


Virtual-Memory Management

Background

- **Virtual memory** – separation of user logical memory from physical memory.
 - Only part of the program needs to be in memory for execution
 - Logical address space can therefore be much larger than physical address space
 - Allows address spaces to be shared by several processes
 - Allows for more efficient process creation
- Virtual memory can be implemented via:
 - Demand paging
 - Demand segmentation

Virtual Memory That is Larger Than Physical Memory



Demand Paging

- Bring a page into memory only when it is needed
 - Less I/O needed
 - Less memory needed
 - Faster response
 - More users
- Page is needed $\Rightarrow$ reference to it
 - invalid reference $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
 - Swapper that deals with pages is a **pager**

Valid-Invalid Bit

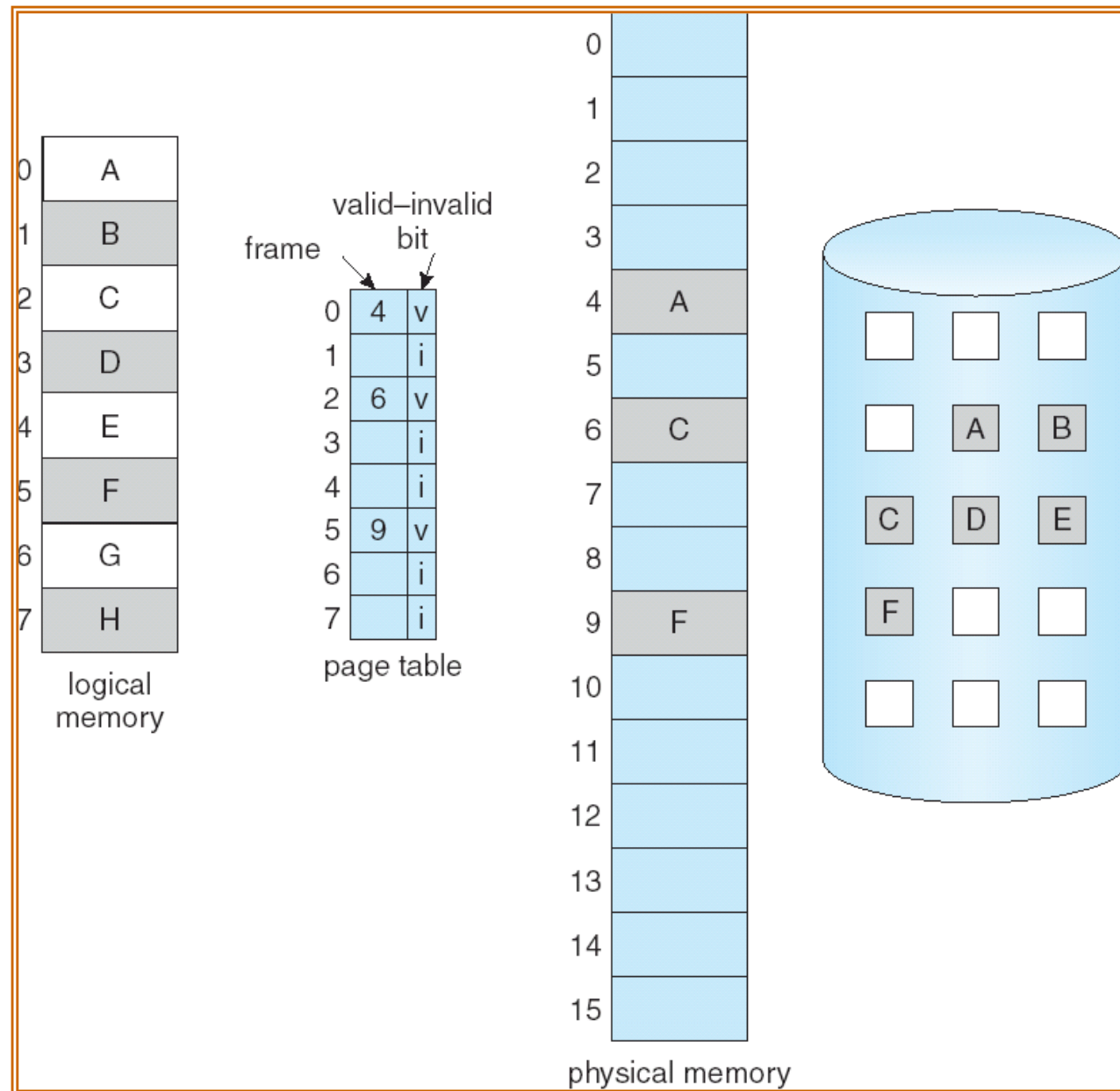
- With each page table entry a valid–invalid bit is associated (**v** $\Rightarrow$ in-memory, **i** $\Rightarrow$ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:

Frame #	valid-invalid bit
	v
	v
	v
	v
	i
....	
	i
	i

page table

- During address translation, if valid–invalid bit in page table entry is **i** $\Rightarrow$ page fault

Page Table When Some Pages Are Not in Main Memory



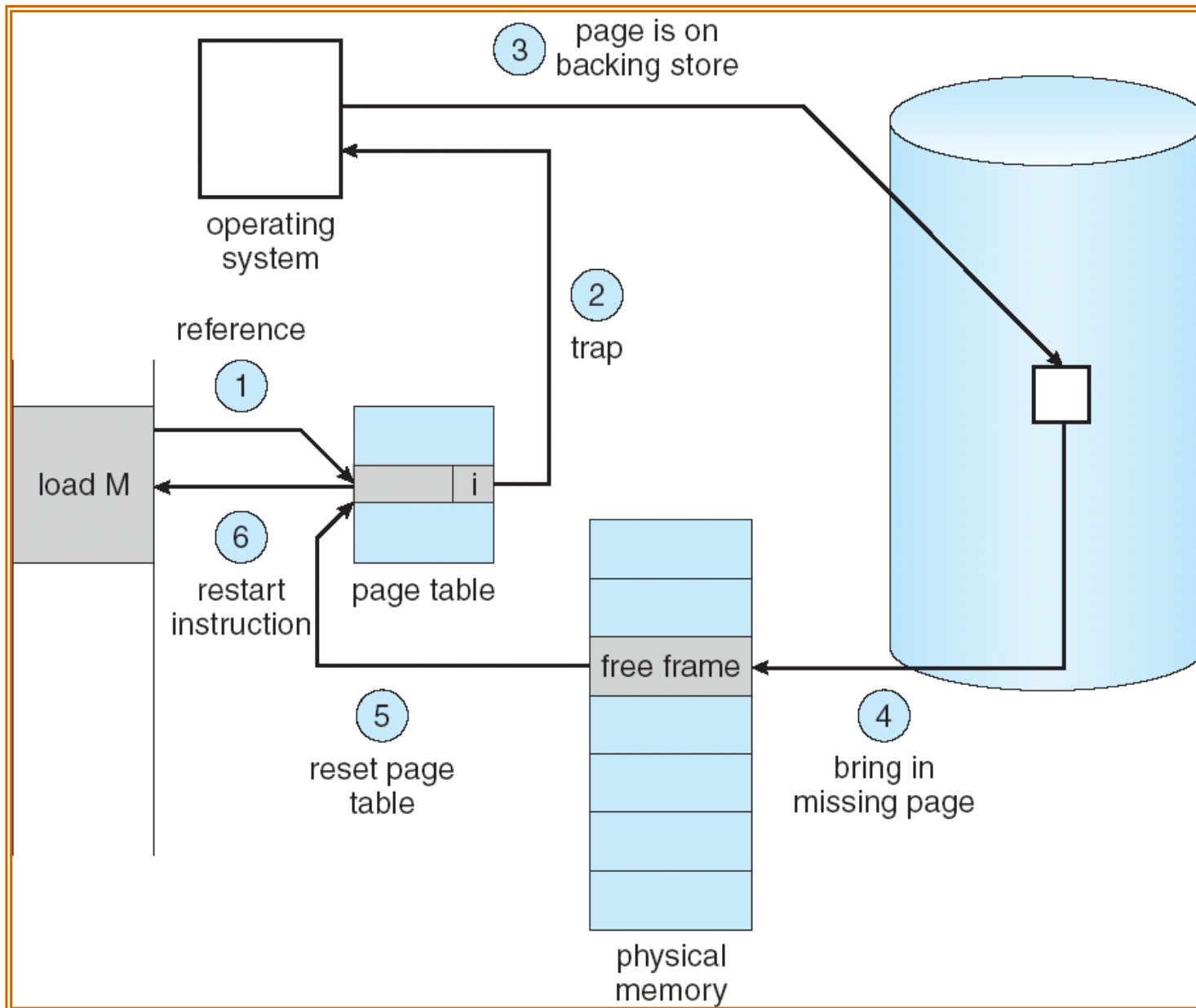
Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:

page fault

1. Operating system looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort
 - Just not in memory
2. Get empty frame
3. Swap page into frame
4. Reset tables
5. Set validation bit = **v**
6. Restart the instruction that caused the page fault

Steps in Handling a Page Fault



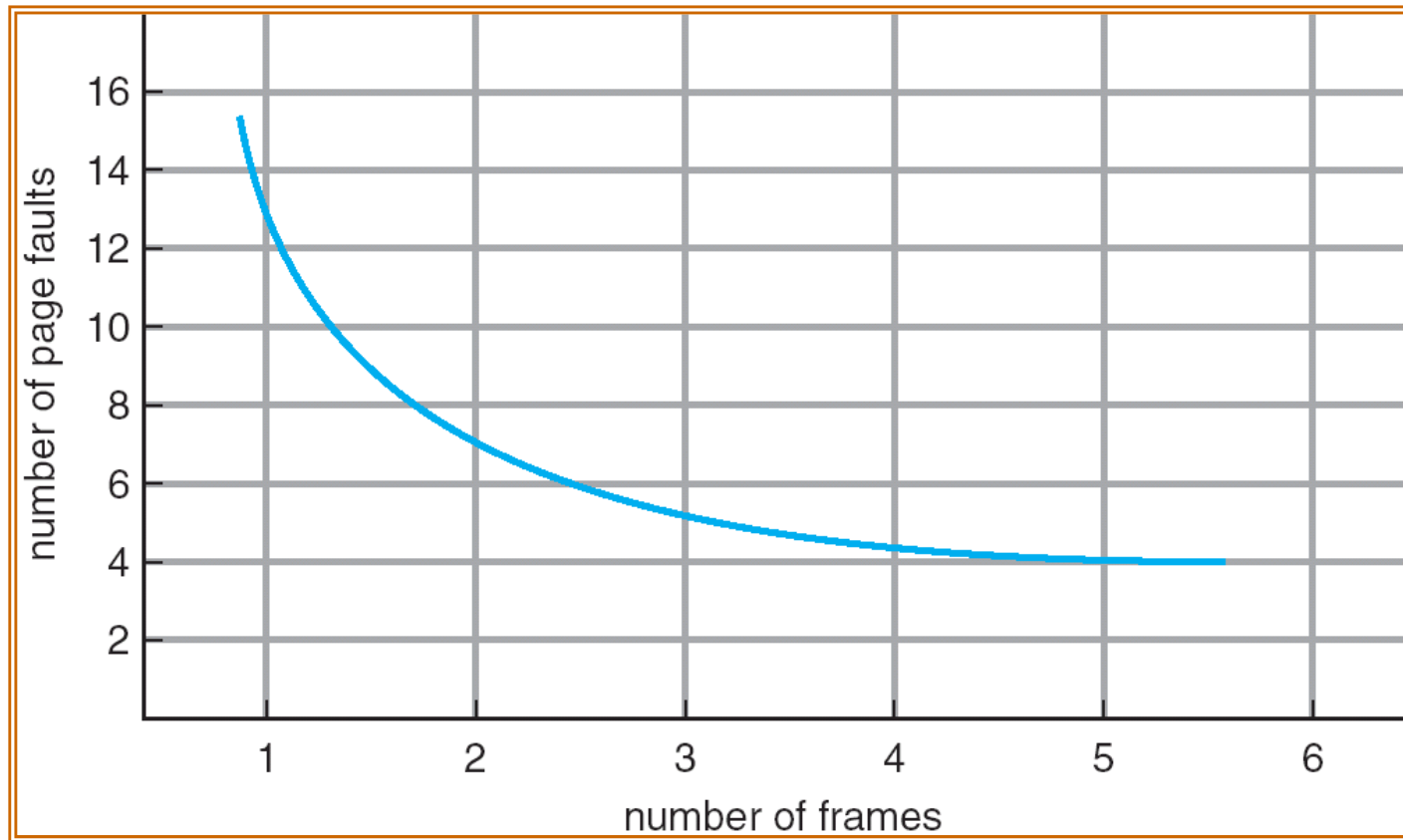
What happens if there is no free frame?

- Page replacement – find some page in memory, but not really in use, swap it out
 - algorithm
 - performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

Page Replacement Algorithms

- Want lowest page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string

Graph of Page Faults Versus The Number of Frames



FIFO Page Replacement

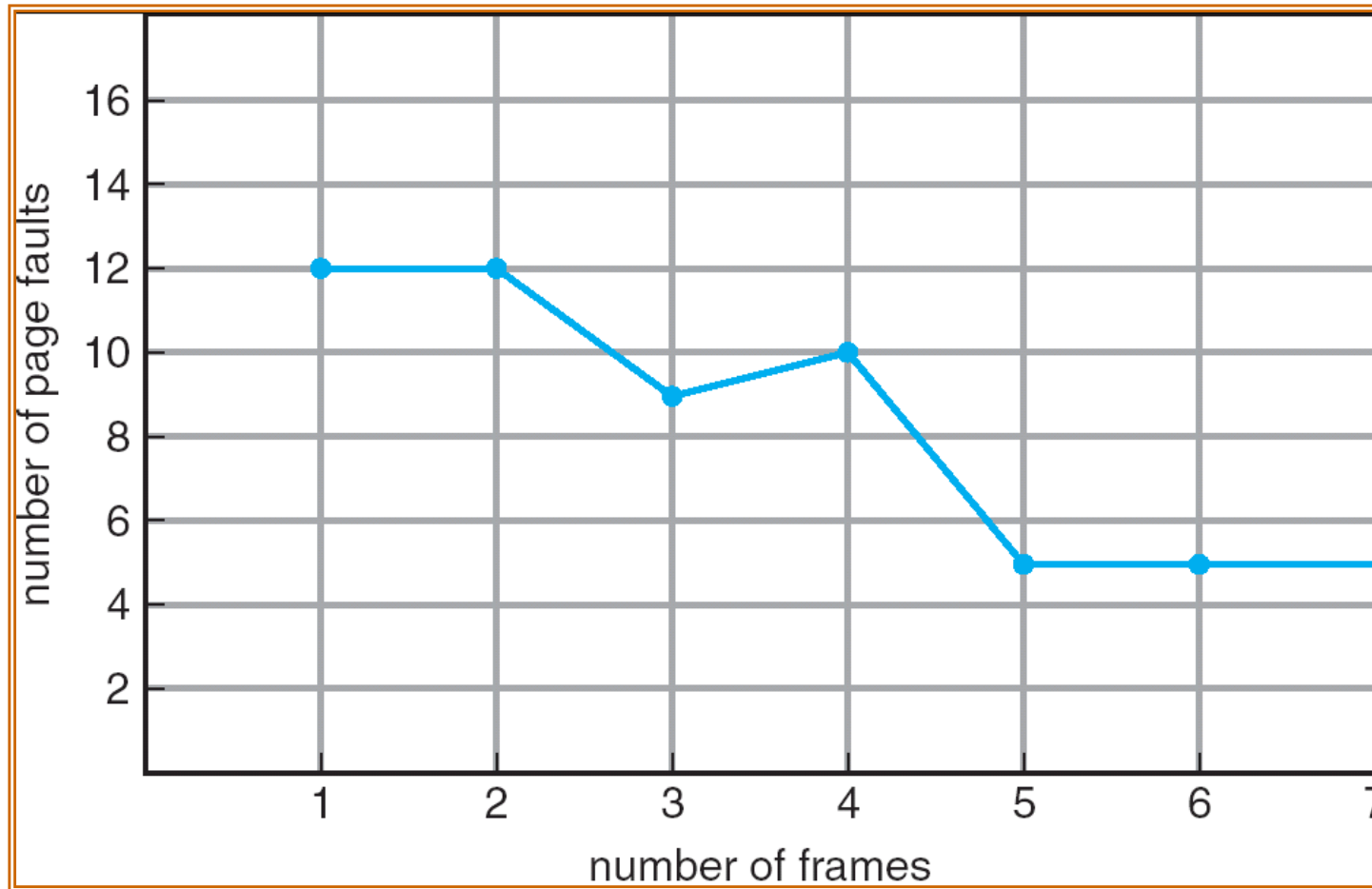
reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0																
		1	1																

page frames

FIFO Illustrating Belady's Anomaly



Optimal Algorithm

- Replace page that will not be used for longest period of time
- How do you know this?
- Used for measuring how well your algorithm performs

Optimal Page Replacement

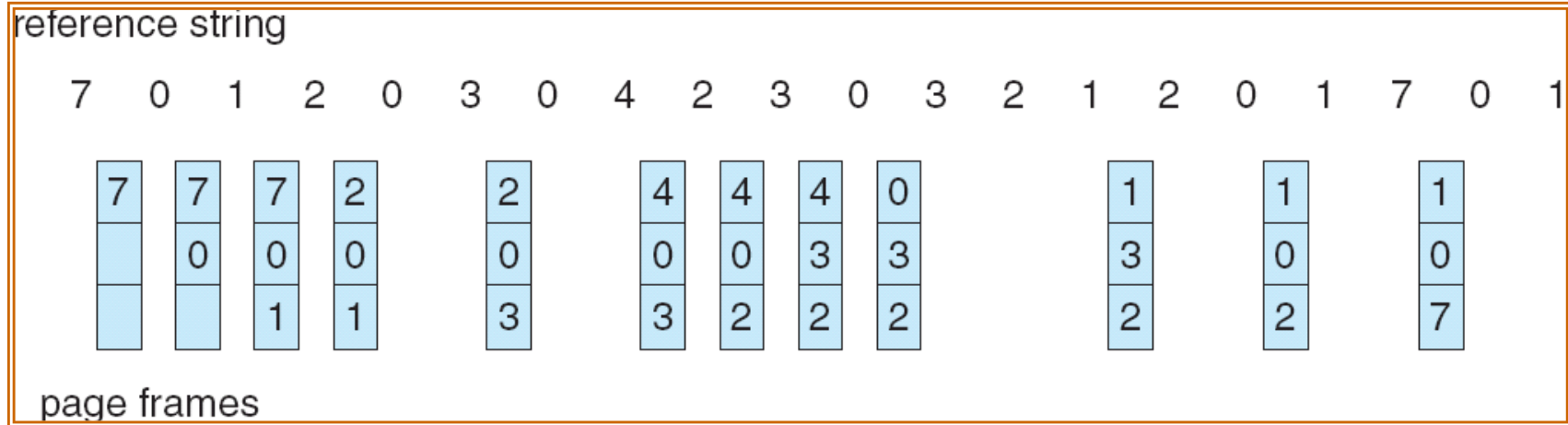
reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2			2			2				7
	0	0	0		0		4			0			0				0
		1	1		3		3			3			1				1

page frames

LRU Page Replacement



Global vs. Local Allocation

- **Global replacement** – process selects a replacement frame from the set of all frames; one process can take a frame from another
- **Local replacement** – each process selects from only its own set of allocated frames

Thrashing

- ❑ **Thrashing** $\equiv$ a process is busy swapping pages in and out
- ❑ If a process does not have “enough” pages, the page-fault rate is very high. This leads to:
 - ❑ low CPU utilization
 - ❑ operating system thinks that it needs to increase the degree of multiprogramming
 - ❑ another process added to the system

Thrashing (Cont.)

- ❑ Operation becomes inefficient
- ❑ Caused when a page is removed from memory but is called back shortly thereafter
- ❑ Can occur across jobs, when a large number of jobs are vying for a relatively few number of free pages
- ❑ Can happen within a job (e.g., in loops that cross page boundaries)

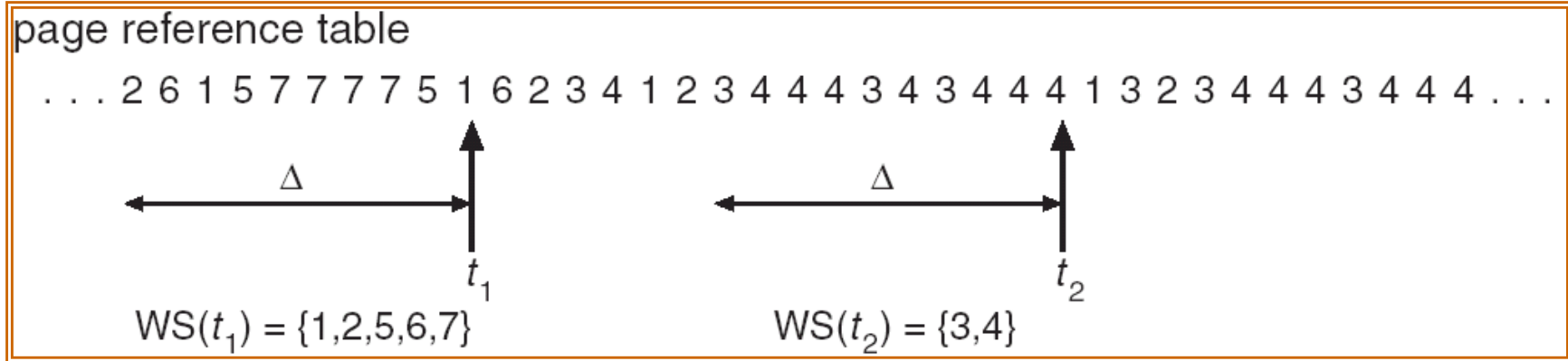
The Working Set

- **Working set:** Set of pages residing in memory that can be accessed directly without incurring a page fault
 - Improves performance of demand page schemes
 - Requires the concept of “locality of reference”
- **System must decide**
 - How many pages compose the working set
 - The maximum number of pages the operating system will allow for a working set

Working-Set Model

- $\Delta \equiv$ working-set window $\equiv$ a fixed number of page references
Example: 10,000 instruction
- WSS_i (working set of Process P_i) =
total number of pages referenced in the most recent Δ (varies in time)
 - if Δ too small will not encompass entire locality
 - if Δ too large will encompass several localities
 - if $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \sum WSS_i \equiv$ total demand frames
- if $D > m \Rightarrow$ Thrashing
- Policy if $D > m$, then suspend one of the processes

Working-set model



Keeping Track of the Working Set

- Approximate with interval timer + a reference bit
- Example: $\Delta = 10,000$
 - Timer interrupts after every 5000 time units
 - Keep in memory 2 bits for each page
 - Whenever a timer interrupts copy and sets the values of all reference bits to 0
 - If one of the bits in memory = 1 $\Rightarrow$ page in working set
- Why is this not completely accurate?
- Improvement = 10 bits and interrupt every 1000 time units

Program Structure

- Program structure

- `int[128,128] data;`

- Each row is stored in one page

- Program 1

```
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i,j] = 0;
```

128 x 128 = 16,384 page faults

- Program 2

```
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i,j] = 0;
```

128 page faults